

y m
yy

Document Details

Submission ID

trn:oid:::1:3534379698

Submission Date

Apr 12, 2026, 1:50 PM GMT+3

Download Date

Apr 12, 2026, 1:58 PM GMT+3

File Name

b30b2e66329040fb8342c48375b09ad8.pdf

File Size

595.7 KB

70 Pages

20,527 Words

106,282 Characters

29% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

-  **66 AI-generated only 29%**
Likely AI-generated text from a large-language model.
-  **0 AI-generated text that was AI-paraphrased 0%**
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely AI generated as AI generated and AI paraphrased or likely AI generated and AI paraphrased writing as only AI generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.





THE DIVIDE-AND-CONQUER ALGORITHM FOR SYMMETRIC EIGENPROBLEMS

Ming Yin

Supervisor: Associate Professor William McLean

School of Mathematics and Statistics
UNSW Sydney

April 2026

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS OF THE DEGREE OF
MASTER OF MATHEMATICS

Plagiarism statement

I declare that this thesis is my own work, except where acknowledged, and has not been submitted for academic credit elsewhere.

I acknowledge that the assessor of this thesis may, for the purpose of assessing it:

- Reproduce it and provide a copy to another member of the University; and/or,
- Communicate a copy of it to a plagiarism checking service (which may then retain a copy of it on its database for the purpose of future plagiarism checking).

I certify that I have read and understood the University Rules in respect of Student Academic Misconduct, and am aware of any potential plagiarism penalties which may apply.

By signing this declaration I am agreeing to the statements and conditions above.

Signed: _____

Date: _____

Acknowledgements

I would like to express my sincere gratitude to my supervisor for the guidance, patience, and support provided throughout the completion of this thesis.

I would also like to thank my roommates and the friends I have made in Australia. Studying abroad can be challenging at times, and their companionship made my experience here both enjoyable and memorable.

Finally, I would like to thank my parents for their support and encouragement throughout my studies.

Abstract

Computing eigenvalues and eigenvectors of real symmetric matrices is a central problem in numerical linear algebra. The symmetry gives a well-structured mathematical problem, but for dense matrices the practical cost of computing all eigenvectors still depends strongly on how the tridiagonal eigenproblem is solved after the initial reduction. This thesis studies that issue through the classical QR approach and the divide-and-conquer approach for symmetric eigenproblems.

The thesis follows the problem from mathematics to implementation and then to experiment. It first reviews Householder reduction to tridiagonal form, implicit QR/QL iteration for symmetric tridiagonal matrices, and the divide-and-conquer strategy based on splitting, secular equations, and merging. It then examines how these ideas appear in LAPACK through the dense symmetric drivers DSYEV and DSYEVD. The key implementation difference is that DSYEV uses the QR-based tridiagonal routine DSTEQR, whereas DSYEVD uses the divide-and-conquer routine DSTEDC.

The experiments show that this tridiagonal-stage difference explains most of the observed runtime gap when full eigenpairs are computed. The shared dense reduction stage has nearly the same cost for the two drivers, while the DSTEDC path is much faster than the DSTEQR path for the tested large cases. Profiling supports the same interpretation: DSTEQR is dominated by fine-grained rotation updates, whereas DSTEDC and the associated back-transformation expose more blocked matrix operations. The practical conclusion is therefore not simply that one driver is universally better. For full eigenpair computation, the divide-and-conquer route gives a clear runtime advantage in the tested setting, but it does so by using substantially more workspace. The choice between the two approaches is best understood as a tradeoff between runtime and memory.

Contents

Chapter 1	Introduction	1
1.1	Applications and Significance of the Eigenvalue Problem	1
1.2	LAPACK and BLAS as the Implementation Context	2
1.3	A Motivating Comparison of DSYEV and DSYEVD	2
1.4	Scope and Project Aims	3
1.5	Organization of the Thesis	3
Chapter 2	Mathematical Foundations of Symmetric Eigenproblems	4
2.1	Householder Tridiagonalization	4
2.1.1	Householder reflector	4
2.1.2	Single similarity step	5
2.1.3	Rank-2 update form	5
2.1.4	Completion	5
2.1.5	Cost	5
2.2	Implicit QR/QL Iteration for Symmetric Tridiagonal Matrices . . .	5
2.2.1	Problem Setting and Iteration Goal	6
2.2.2	Spectral Decomposition and Subspace Convergence	6
2.2.3	Block Iteration and the QR Iteration	7
2.2.4	Shifted QR Iteration	8
2.2.5	Symmetric Tridiagonal Case and Deflation	8
2.2.6	Wilkinson Shift	9
2.2.7	Implicit Shift, Bulge Generation, and Bulge Chasing	10
2.2.8	Implicit Q Theorem and Equivalence to Explicit Shifted QR	12
2.2.9	One Francis Sweep (Algorithmic Form)	14
2.2.10	Worked Example on a Common Test Matrix	15
2.2.11	Practical Remarks	19
2.3	Divide-and-Conquer Method for Tridiagonal Eigenproblems	19
2.3.1	Motivation and Roadmap	19
2.3.2	Divide Step: Splitting into Compensated Blocks	20
2.3.3	Subproblem Diagonalization and Recursive Structure	22
2.3.4	Merge Step: Diagonal-Plus-Rank-One Form	23
2.3.5	Secular Equation	24
2.3.6	Eigenvector Reconstruction	27
2.3.7	Deflation and Reordering	27
2.3.8	Complexity and Parallel Structure	28
2.3.9	Summary	28
2.4	Comparison of Algorithmic Procedures	29
2.4.1	Conceptual Comparison	29

2.4.2	Complexity and Numerical Stability	30
2.4.3	Conclusion	30
Chapter 3	Implementation-Level Comparison of DSYEV and DSYEVD	31
3.1	Overview of the Driver Structure	31
3.1.1	Interface-Level View	31
3.1.2	Common Driver Pipeline	32
3.1.3	Main Points of Comparison	32
3.2	Key Implementation Blocks	32
3.2.1	SYTRD: Householder Tridiagonalization in LAPACK	32
3.2.2	STEQR: Implicit QR/QL Iteration for Tridiagonal Matrices	33
3.2.3	STEDC: Divide-and-Conquer Implementation in LAPACK	34
3.3	Implementation Implications	36
Chapter 4	Experimental Evaluation and Performance Analysis	38
4.1	Experimental Setup	38
4.1.1	Hardware Platform	38
4.1.2	Software Configuration	39
4.1.3	Test Matrices	39
4.2	Overall Performance Benchmark	40
4.2.1	Single-Thread Performance	40
4.2.2	Thread Scalability	41
4.3	Impact of the Netlib-to-OpenBLAS Implementation Change	41
4.3.1	Stage-by-Stage Comparison	42
4.4	Impact of SVE Vectorization	42
4.4.1	Stage-by-Stage SIMD Speedup	42
4.5	Detailed Performance Profiling	43
4.5.1	Analysis of DSTEQR	44
4.5.2	Analysis of DSTEDC	44
4.5.3	Hardware-Efficiency Evidence	45
4.6	Final Synthesis and Implications	47
Chapter 5	Conclusion and Future Work	49
5.1	Main Findings	49
5.2	Practical Guidance for Routine Selection	50
5.3	Limitations of This Study	50
5.4	Future Work	51
Appendix A	Summary of Main LAPACK Routines	52
Appendix B	Detailed Experimental Setup	53
B.1	Hardware and Software Configuration	53
B.2	Measurement Protocol	53
B.3	Additional Profiling Trees	54
B.4	Test Matrices and Correctness Checks	55
References		57

Acknowledgement of AI Use

58

CHAPTER 1

Introduction

This thesis studies numerical methods for computing eigenvalues and eigenvectors of dense real symmetric matrices. A standard dense symmetric eigensolver follows the three-stage route shown in Figure 1.1. First, an orthogonal similarity transformation reduces the dense matrix A to a symmetric tridiagonal matrix T . Second, a tridiagonal eigensolver computes the eigenvalues and tridiagonal eigenvectors. Third, when eigenvectors of the original dense matrix are required, they are recovered by a back transformation. The focus of this thesis is on two classical choices for the second stage: QR iteration and divide-and-conquer. Both approaches are mathematically well established, yet their computational behaviour can be quite different when all eigenvectors are required.

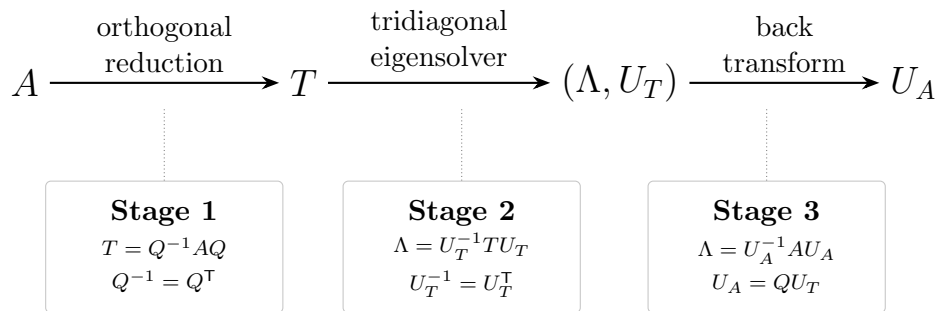


Figure 1.1: Standard three-stage route for the dense real symmetric eigenproblem.

The comparison is developed from three viewpoints. First, the thesis reviews the mathematical structure of the two tridiagonal eigensolvers. Second, it examines how the same ideas are realized in practical dense linear algebra software. Third, it tests how the different algorithmic structures appear in measured runtime. LAPACK's DSYEV and DSYEVD routines provide the concrete implementation setting for this comparison, but the underlying question is the algorithmic contrast between QR-based and divide-and-conquer methods.

1.1 Applications and Significance of the Eigenvalue Problem

Real symmetric eigenvalue problems occupy a central place in numerical linear algebra because symmetry gives a particularly clean spectral structure: the eigenvalues are real and the eigenvectors may be chosen orthogonally. In applications this structure is not only mathematically convenient but also physically meaningful. Typical examples include vibration models, structural mechanics, and discretized stability problems, where eigenvalues may represent frequencies, modes, or stability thresholds, and eigenvectors describe the corresponding directions or shapes.

For small dense problems, the choice of eigensolver may not matter much beyond software convenience. For larger dense problems, however, the cost of forming all eigenvectors becomes substantial, and implementation details begin to matter. Classical QR-based methods are robust and well established [1, 2], but divide-and-conquer methods [3, 4] reorganize the work in a way that is often more compatible with modern BLAS-based implementations [5]. This is the practical motivation for the thesis: two mathematically equivalent routes can ask the computer to do rather different kinds of work.

The thesis therefore focuses on a specific comparison rather than a broad survey. It develops the mathematics behind the QR and divide-and-conquer paths, then traces how those paths appear in a standard implementation setting, and finally checks the consequences experimentally.

1.2 LAPACK and BLAS as the Implementation Context

The mathematical algorithms studied in this thesis are implemented in practice through numerical linear algebra libraries. In dense linear algebra, LAPACK provides standard high-level routines, while BLAS supplies the lower-level vector and matrix kernels on which many LAPACK routines depend [2]. This makes LAPACK a natural implementation context for comparing QR-based and divide-and-conquer eigensolvers.

This context is important because practical performance is not determined only by the asymptotic operation count of an algorithm. It also depends on how the work is organized. Computations that can be expressed through blocked matrix operations can make better use of optimized BLAS kernels and memory hierarchy, whereas algorithms dominated by fine-grained sequential updates may benefit less from the same hardware and library support.

For dense symmetric eigenproblems, LAPACK exposes this contrast through the drivers `DSYEV` and `DSYEVD`. Both follow the standard three-stage route shown in Figure 1.1: reduction to tridiagonal form, solution of the tridiagonal eigenproblem, and back transformation of eigenvectors when required. Their main difference lies in the second stage. `DSYEV` follows the QR-based tridiagonal path, whereas `DSYEVD` uses divide-and-conquer. Thus LAPACK provides a concrete setting in which the mathematical difference between the two eigensolvers can be connected to measured computational behaviour.

1.3 A Motivating Comparison of `DSYEV` and `DSYEVD`

The LAPACK routines provide a convenient motivating example because they solve the same dense symmetric problem and expose closely related interfaces. `DSYEV` follows the classical QR-based tridiagonal path, whereas `DSYEVD` uses divide-and-conquer. This makes them a useful concrete pair for seeing how the two mathematical approaches behave in practice.

Table 1.1 reports a short comparison for both the eigenvalues-only case and the full eigenpair case. The matrix sizes are chosen to align with the later benchmark chapter. The pattern is already visible here. When only eigenvalues are requested, the two routines are nearly identical in runtime across the tested sizes. When eigenvectors are also requested, the gap is much larger: in this preview, `DSYEVD` is about $4\times$ faster at $n = 512$ and about $5.4\times$ faster at $n = 4096$.

For the present thesis, this table is only a starting point. Its role is to show that the practical difference is not hypothetical and to motivate a more careful question: which stage of the computation is responsible for that gap, and what feature of the two algorithms creates it? Chapter 4 returns to this comparison under a controlled experimental setup and answers that question in detail.

Table 1.1: Timing comparison of DSYEV and DSYEVD.

Matrix size n	DSYEV (QR)		DSYEVD (D&C)	
	Eigvals [s]	Eigpairs [s]	Eigvals [s]	Eigpairs [s]
512	0.030	0.192	0.030	0.048
1024	0.230	1.427	0.232	0.323
2048	1.849	12.557	1.931	2.399
4096	17.294	106.606	17.208	19.804

1.4 Scope and Project Aims

The scope is restricted to dense real symmetric eigenproblems and to the comparison between QR-based iteration and divide-and-conquer at the tridiagonal stage. The aim is not to survey all symmetric eigensolvers or all LAPACK routines, but to understand one representative algorithmic contrast in some detail. The LAPACK routines DSYEV and DSYEVD are used as concrete implementations of the two approaches.

Within this scope, the thesis has three aims:

- to explain how QR iteration and divide-and-conquer solve the tridiagonal eigenproblem differently;
- to trace how this difference appears in the LAPACK paths associated with DSYEV and DSYEVD;
- to measure where the runtime difference comes from and discuss the resulting runtime–workspace tradeoff.

1.5 Organization of the Thesis

The remainder of the thesis follows this sequence. Chapter 2 develops the mathematical background: Householder tridiagonalization, QR/QL iteration, and divide-and-conquer for tridiagonal eigenproblems. Chapter 3 connects these algorithms to their LAPACK realizations. Chapter 4 presents the experimental study and identifies which stages account for the measured runtime differences. Chapter 5 summarizes the conclusions, practical guidance, limitations, and possible extensions.

CHAPTER 2

Mathematical Foundations of Symmetric Eigenproblems

This chapter develops the mathematical background for dense real symmetric eigenvalue problems. The starting point is the spectral theorem: a real symmetric matrix admits real eigenvalues and an orthonormal basis of eigenvectors. Computationally, the dense problem is first reduced by orthogonal similarity transformations to a symmetric tridiagonal problem, after which the remaining task is to diagonalize that tridiagonal matrix.

The chapter follows this mathematical pipeline. Section 2.1 derives Householder tridiagonalization. Section 2.2 presents implicit QR/QL iteration for symmetric tridiagonal matrices, including shifts, deflation, and bulge chasing. Section 2.3 develops the divide-and-conquer method through splitting, rank-one correction, the secular equation, and eigenvector reconstruction. Section 2.4 compares these two tridiagonal eigensolvers in terms of their computational structure, cost, and numerical behaviour.

2.1 Householder Tridiagonalization

Given a real symmetric matrix $A \in \mathbb{R}^{n \times n}$, the goal is to construct an orthogonal matrix Q such that

$$T = Q^T A Q$$

is symmetric and tridiagonal. Since orthogonal similarity preserves eigenvalues, this reduction transforms the symmetric eigenvalue problem into a tridiagonal one without altering its spectrum. This method traces back to Householder and was formalised for the symmetric case by Martin–Reinsch–Wilkinson (1968). We write $A^{(j)}$ for the matrix after the j -th similarity step and use indices i for rows/columns; this avoids mixing iteration counters with position indices.

2.1.1 Householder reflector

Definition 2.1.1 (Householder reflector). *For a vector $x \in \mathbb{R}^m$, define*

$$\alpha = \text{sign}(x_1)\|x\|_2, \quad v = x + \alpha e_1, \quad H = I - 2 \frac{vv^T}{v^T v}.$$

Then H is orthogonal and symmetric, and

$$Hx = \begin{bmatrix} -\alpha \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

So this Householder reflector eliminates all but the first entry of x .

2.1.2 Single similarity step

Let $A^{(1)} = A$. At step j ($1 \leq j \leq n - 2$), consider the subvector

$$x = A_{j+1:n,j}^{(j)}.$$

Construct a Householder reflector H_j such that $H_j x = (\pm \|x\|, 0, \dots, 0)^T$, and embed it into an $n \times n$ orthogonal matrix by

$$Q_j = \text{diag}(I_j, H_j).$$

Define

$$A^{(j+1)} = Q_j^T A^{(j)} Q_j.$$

Then all entries $A_{i,j}^{(j+1)}$ with $i \geq j + 2$ vanish, and the symmetry of $A^{(j)}$ guarantees that the same entries in column j and row j are eliminated simultaneously.

2.1.3 Rank-2 update form

Let u denote the Householder vector corresponding to H_j . The similarity update on the trailing block admits the explicit rank-2 expression

$$A^{(j+1)} = A^{(j)} - u y^T - y u^T, \quad y = A^{(j)} u - \frac{1}{2}(u^T A^{(j)} u) u. \quad (\star)$$

Equation $(\star)$ is the classical symmetric Householder reduction formula.

2.1.4 Completion

Repeating the above construction for $j = 1, \dots, n - 2$ yields

$$T = A^{(n-1)} = Q^T A Q, \quad Q = Q_1 Q_2 \cdots Q_{n-2},$$

where T is symmetric and tridiagonal. Since Q is orthogonal, T and A share all eigenvalues.

2.1.5 Cost

At step j , the trailing block has dimension $m = n - j$. Forming the reflector costs $O(m)$ flops, while applying the rank-2 update dominates with $\approx 4m^2$ flops. Therefore,

$$\sum_{j=1}^{n-2} 4(n-j)^2 = \frac{4}{3}n^3 + O(n^2),$$

so Householder tridiagonalization requires

$$\frac{4}{3}n^3 + O(n^2) \text{ flops.}$$

2.2 Implicit QR/QL Iteration for Symmetric Tridiagonal Matrices

2.2.1 Problem Setting and Iteration Goal

Consider the symmetric tridiagonal eigenvalue problem

$$T = \begin{bmatrix} a_1 & b_1 & & & \\ b_1 & a_2 & b_2 & & \\ & b_2 & \ddots & \ddots & \\ & & \ddots & a_{n-1} & b_{n-1} \\ & & & b_{n-1} & a_n \end{bmatrix}$$

arising from the Householder reduction in Section 2.1. Equivalently, starting from a dense symmetric matrix A , the previous stage computes an orthogonal matrix X_0 such that

$$A_0 = X_0^T A X_0,$$

where A_0 is symmetric tridiagonal. The task is then to diagonalize this tridiagonal matrix A_0 by orthogonal similarity transformations, working block by block on its active unreduced part. The goal is to compute all eigenvalues of T , and optionally its orthonormal eigenvectors, by a sequence of orthogonal similarity transformations. For exposition, this section adopts the QR viewpoint; the same tridiagonal framework may also be formulated as an implicit QL iteration on the current active block. In either form, the subdiagonal entries $b_i^{(k)}$ are driven toward zero; once a subdiagonal element is sufficiently small, the matrix decouples into blocks and the associated eigenvalue (or cluster) is isolated.

This section follows a single line of ideas: spectral decomposition $\rightarrow$ block iteration $\rightarrow$ QR iteration $\rightarrow$ shifted QR $\rightarrow$ tridiagonal deflation $\rightarrow$ Wilkinson shift $\rightarrow$ implicit bulge chasing $\rightarrow$ Implicit Q theorem $\rightarrow$ Francis sweep. Its purpose is to show how these ideas lead to the implicit shifted QR/QL iteration for symmetric tridiagonal matrices.

Notation: The symbol k denotes iteration indices, i denotes row/column indices. For a tridiagonal matrix, a_i denotes diagonal entries and b_i denotes subdiagonal entries.

2.2.2 Spectral Decomposition and Subspace Convergence

Let $A \in \mathbb{R}^{n \times n}$ be symmetric. Then it admits an orthogonal spectral decomposition

$$A = V \Lambda V^T,$$

where

$$V = [v_1, \dots, v_n], \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_n),$$

with

$$A v_j = \lambda_j v_j, \quad v_j^T v_k = \delta_{jk}, \quad |\lambda_1| \geq |\lambda_2| \geq \dots \geq |\lambda_n|.$$

For $1 \leq m \leq n-1$, define the invariant subspace

$$E_m := \text{span}\{v_1, \dots, v_m\}.$$

We first consider the action of repeated powers on a single vector. Let

$$x = \sum_{j=1}^n c_j v_j, \quad c_j = v_j^\top x.$$

Then

$$A^k x = \sum_{j=1}^n c_j \lambda_j^k v_j.$$

Thus the spectral magnitudes determine which components dominate under repeated application of A .

Let $S_m \subset \mathbb{R}^n$ be an arbitrary m -dimensional subspace. Let $X \in \mathbb{R}^{n \times m}$ be an orthonormal basis matrix whose columns span S_m . Expanding X in the eigenbasis of A , write

$$X = VC, \quad C \in \mathbb{R}^{n \times m}.$$

Repeated application of A yields

$$A^k X = V \Lambda^k C.$$

If $|\lambda_m| > |\lambda_{m+1}|$, then after normalization or orthogonalization the column space of $A^k X$ is driven toward the invariant subspace

$$E_m = \text{span}\{v_1, \dots, v_m\}.$$

Thus repeated application of A tends to amplify the dominant eigenspace components of a generic m -dimensional subspace, providing the spectral motivation for subspace iteration, block iteration, and ultimately QR iteration [6, 7].

2.2.3 Block Iteration and the QR Iteration

Starting from block subspace iteration and orthogonalization at each step, we obtain the QR form $A_{k-1} = Q_k R_k$, $A_k = R_k Q_k$. The orthogonalization is essential: without it, repeated multiplication by A would drive basis vectors toward the dominant eigendirections and lead to loss of linear independence [6].

Let $X_0 = [x_1^{(0)} \cdots x_n^{(0)}]$ be orthogonal. One block-iteration step is

$$AX_{k-1} = X_k R_k,$$

where X_k is orthogonal and R_k is upper triangular. If one were to iterate only with $A^k X_0$ and never reorthogonalize, then the columns would progressively align with the dominant eigendirections and the basis would lose linear independence. The QR step prevents this collapse by replacing the multiplied basis with an orthonormal one at every iteration while retaining the same column space. Define

$$A_k := X_k^\top A X_k.$$

Then

$$A_{k-1} = X_{k-1}^\top A X_{k-1} = X_{k-1}^\top X_k R_k = Q_k R_k, \quad Q_k := X_{k-1}^\top X_k,$$

and

$$A_k = X_k^T A X_k = X_k^T A X_{k-1} X_{k-1}^T X_k = X_k^T X_k R_k X_{k-1}^T X_k = R_k Q_k.$$

$$A_{k-1} = Q_k R_k, \quad A_k = R_k Q_k,$$

i.e., a sequence of orthogonal similarity transforms. Therefore the orthogonalized block iteration produces exactly the QR iteration [6, 8]. Since

$$A_k = Q_k^T A_{k-1} Q_k,$$

each step is an orthogonal similarity transformation and hence preserves the eigenvalues. This identity is the algebraic starting point for the shifted QR iteration used in STEQR.

2.2.4 Shifted QR Iteration

The unshifted QR iteration already preserves eigenvalues, but its convergence may be slow in practice. The standard acceleration is to introduce a shift μ_k and apply QR not to A_{k-1} itself, but to $A_{k-1} - \mu_k I$.

Introduce a shift μ_k and factor

$$A_{k-1} - \mu_k I = Q_k R_k.$$

Then update

$$A_k = \mu_k I + R_k Q_k.$$

Using the factorization above,

$$A_k = \mu_k I + R_k Q_k \tag{2.2.1}$$

$$= \mu_k I + Q_k^T (A_{k-1} - \mu_k I) Q_k \tag{2.2.2}$$

$$= Q_k^T A_{k-1} Q_k. \tag{2.2.3}$$

Therefore the shift does not alter the eigenvalue problem: each step remains an orthogonal similarity transformation. Its role is to accelerate QR iteration by choosing μ_k close to a target eigenvalue [9, 10, 6]. For symmetric tridiagonal matrices, a good shift typically speeds the decay of the trailing subdiagonal and hence accelerates deflation.

2.2.5 Symmetric Tridiagonal Case and Deflation

Let A_0 be symmetric tridiagonal. In the QR viewpoint adopted here, shifted QR is applied blockwise to active unreduced tridiagonal blocks. After each sweep, test

$$|b_i^{(k)}| \leq \tau (|a_i^{(k)}| + |a_{i+1}^{(k)}|),$$

with a machine-scaled threshold τ . If true, set $b_i^{(k)} = 0$, split the matrix into independent blocks, and continue on each block. In the QR-oriented form of the iteration, the active block is viewed from the bottom upward: the sweep is designed to drive the trailing subdiagonal toward zero, so that the bottom-right part decouples first. Thus the practical iteration repeatedly acts on the current unreduced trailing principal block, shrinks that block after deflation, and continues until only

diagonal entries remain $[2, 7]$. Recall that an upper Hessenberg matrix H satisfies $H_{ij} = 0$ for $i > j + 1$; a symmetric tridiagonal matrix is a special case.

Proposition 2.2.1 (Preservation of tridiagonal structure). *Suppose A_{k-1} is symmetric tridiagonal and*

$$A_{k-1} - \mu_k I = Q_k R_k, \quad A_k = \mu_k I + R_k Q_k.$$

Then A_k is again tridiagonal.

This standard Hessenberg/tridiagonal invariance of shifted QR is discussed in Golub–Van Loan and Parlett [6, 7].

Proof. Let $\mathcal{L}(M)$ denote the lower bandwidth of a matrix M . Since A_{k-1} is tridiagonal, $\mathcal{L}(A_{k-1}) = 1$. Subtracting $\mu_k I$ does not change bandwidth, so

$$\mathcal{L}(A_{k-1} - \mu_k I) = 1.$$

For the QR factorization of a Hessenberg or tridiagonal matrix, the orthogonal factor satisfies

$$\mathcal{L}(Q_k) \leq \mathcal{L}(A_{k-1} - \mu_k I) = 1,$$

while R_k is upper triangular, hence

$$\mathcal{L}(R_k) = 0.$$

For any $n \times n$ matrices A, B , the lower-bandwidth bound

$$\mathcal{L}(AB) \leq \mathcal{L}(A) + \mathcal{L}(B)$$

holds: from

$$(AB)_{ij} = \sum_t A_{it} B_{tj},$$

if $(AB)_{ij} \neq 0$, then for some t we have $A_{it} B_{tj} \neq 0$, so

$$i - t \leq \mathcal{L}(A), \quad t - j \leq \mathcal{L}(B),$$

and therefore

$$i - j = (i - t) + (t - j) \leq \mathcal{L}(A) + \mathcal{L}(B).$$

Therefore

$$\mathcal{L}(A_k) = \mathcal{L}(\mu_k I + R_k Q_k) = \mathcal{L}(R_k Q_k) \leq \mathcal{L}(R_k) + \mathcal{L}(Q_k) \leq 1.$$

Because A_k is symmetric and has lower bandwidth at most 1, it is tridiagonal. $\square$

2.2.6 Wilkinson Shift

In the QR-oriented form of the iteration, for the trailing 2×2 block

$$\begin{bmatrix} a & b \\ b & c \end{bmatrix}, \quad \delta := \frac{a - c}{2},$$

the Wilkinson shift is the eigenvalue closer to c , evaluated stably as

$$\mu = c - \text{sign}(\delta) \frac{b^2}{|\delta| + \sqrt{\delta^2 + b^2}},$$

with $\text{sign}(0) = 1$. This shift, commonly called the Wilkinson shift, is the standard practical choice in QR methods for symmetric tridiagonal matrices [11, 7].

2.2.7 Implicit Shift, Bulge Generation, and Bulge Chasing

Instead of explicitly forming and factoring $A_0 - \mu I$, the shift is injected implicitly through the first column

$$x := (A_0 - \mu I)e_1 = \begin{bmatrix} d_1 - \mu \\ e_1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

Because only the first two entries of x are nonzero, a single Givens rotation G_1 , acting on indices $(1, 2)$, is sufficient to map this two-vector to $(\rho, 0)^\top$. Set

$$A_0^{(1)} = G_1^\top A_0 G_1.$$

This first similarity preserves symmetry, but it no longer preserves exact tridiagonality: a new fill-in appears at $(3, 1)$ and, by symmetry, at $(1, 3)$. This isolated out-of-band entry is the *bulge*.

The local 3×3 pattern can be sketched as

$$\begin{bmatrix} \times & \times & \cdot \\ \times & \times & \times \\ \cdot & \times & \times \end{bmatrix} \xrightarrow{G_1^\top(\cdot)G_1} \begin{bmatrix} + & + & + \\ + & + & + \\ + & + & + \end{bmatrix},$$

where $\times$ denotes a known nonzero entry, $\cdot$ denotes a zero entry, $+$ denotes an entry modified by the current similarity update, and $+$ denotes the newly created bulge entry. The symbol $\times$ indicates only structural nonzero positions before the update; its numerical value may still change under the similarity transformation.

The remaining rotations are chosen only to move and eliminate this bulge while keeping the rest of the matrix tridiagonal. For $j = 2, \dots, n-1$, choose G_j on $(j, j+1)$ so that

$$A_0^{(j)} = G_j^\top A_0^{(j-1)} G_j.$$

Because G_j acts only on rows and columns j and $j+1$, it leaves the first basis vector unchanged:

$$G_j e_1 = e_1, \quad e_1^\top G_j = e_1^\top, \quad j \geq 2.$$

Hence, for $j \geq 2$,

$$e_1^\top A_0^{(j)} e_1 = e_1^\top G_j^\top A_0^{(j-1)} G_j e_1 = e_1^\top A_0^{(j-1)} e_1,$$

and likewise

$$A_0^{(j)} e_1 = G_j^T A_0^{(j-1)} e_1.$$

Thus $G_2, G_3, \dots$ do not create any new coupling with e_1 ; they only remove the current bulge and move it one position farther down. Thus $A_0^{(1)}$ has a bulge at $(3, 1)$ and, by symmetry, at $(1, 3)$. Applying G_2 gives

$$A_0^{(2)} = G_2^T A_0^{(1)} G_2,$$

which removes the bulge at $(3, 1)$ and creates the next bulge at $(4, 2)$ and $(2, 4)$. Applying G_3 gives

$$A_0^{(3)} = G_3^T A_0^{(2)} G_3,$$

which moves the bulge further to $(5, 3)$ and $(3, 5)$. In general, the bulge pair moves in the pattern

$$(3, 1) \rightarrow (4, 2) \rightarrow (5, 3) \rightarrow \dots \rightarrow (n, n-2),$$

with the symmetric counterpart in the upper triangle moving simultaneously. The first stages can be visualized as

$$\begin{aligned} A_0^{(1)} &\sim \begin{bmatrix} \times & \times & + & \cdot & \cdot \\ \times & \times & \times & \cdot & \cdot \\ + & \times & \times & \times & \cdot \\ \cdot & \cdot & \times & \times & \times \\ \cdot & \cdot & \cdot & \times & \times \end{bmatrix} \\ &\Downarrow \\ A_0^{(2)} &\sim \begin{bmatrix} \times & \times & \cdot & \cdot & \cdot \\ \times & \times & \times & + & \cdot \\ \cdot & \times & \times & \times & \cdot \\ \cdot & + & \times & \times & \times \\ \cdot & \cdot & \cdot & \times & \times \end{bmatrix} \\ &\Downarrow \\ A_0^{(3)} &\sim \begin{bmatrix} \times & \times & \cdot & \cdot & \cdot \\ \times & \times & \times & \cdot & \cdot \\ \cdot & \times & \times & \times & + \\ \cdot & \cdot & \times & \times & \times \\ \cdot & \cdot & + & \times & \times \end{bmatrix}, \end{aligned}$$

where each $+$ marks the current out-of-band fill-in and its symmetric counterpart. For an active block of order p , after $p-1$ rotations the bulge exits the block and the matrix is tridiagonal again. This is the essential reason the implicit shifted QR step is efficient: the shift is incorporated without forming a dense intermediate matrix [9, 10, 8].

2.2.8 Implicit Q Theorem and Equivalence to Explicit Shifted QR

Definition 2.2.2 (Unreduced Hessenberg matrix). *An upper Hessenberg matrix $H \in \mathbb{R}^{n \times n}$ is called unreduced if every subdiagonal entry is nonzero, that is,*

$$H_{i+1,i} \neq 0, \quad i = 1, \dots, n-1.$$

In the present tridiagonal- QR context, this means the current active block has not yet deflated: every subdiagonal entry in that block is nonzero. If some subdiagonal entry $H_{i+1,i}$ vanishes, then H is called *reduced*; the matrix splits at that position into smaller Hessenberg blocks. The unreduced assumption is needed because the Implicit Q theorem asserts uniqueness of the Hessenberg-preserving orthogonal transformation from its first column, and this uniqueness can fail once such a split occurs. For the present shifted- QR step, the matrix to which the theorem is applied is the shifted active block

$$H := A_0 - \mu I.$$

We only need the version of the theorem tailored to this situation: both the explicit QR factorization and the implicit bulge-chasing step act on the same unreduced Hessenberg matrix H , and the theorem says that once their first column agrees, the whole orthogonal factor is fixed up to signs.

Theorem 2.2.3 (Implicit Q theorem for the shifted block [8, 7]). *Let $H = A_0 - \mu I$ be unreduced upper Hessenberg, and let $\widehat{Q} = [\hat{q}_1, \dots, \hat{q}_n]$ be orthogonal such that $\widehat{Q}^T H \widehat{Q}$ is also unreduced upper Hessenberg. Then $\widehat{Q}$ is determined uniquely by its first column $\hat{q}_1$, up to columnwise signs.*

To apply this to the QR step, first consider the explicit shifted factorization

$$A_0 - \mu I = Q_e R_e,$$

where $Q_e = [q_1^{(e)}, \dots, q_n^{(e)}]$ is orthogonal and R_e is upper triangular. Since $R_e e_1 = (r_{11}, 0, \dots, 0)^T$, we have

$$(A_0 - \mu I)e_1 = Q_e R_e e_1 = r_{11} q_1^{(e)}.$$

Hence the first column of the explicit QR factor is

$$q_1^{(e)} = \pm \frac{(A_0 - \mu I)e_1}{\|(A_0 - \mu I)e_1\|_2}.$$

This is exactly the first-column vector appearing in the theorem. In the present application, the theorem's $\hat{q}_1$ is the normalized vector $(A_0 - \mu I)e_1 / \|(A_0 - \mu I)e_1\|_2$, namely the first column $q_1^{(e)}$ of the explicit QR factor Q_e , up to sign.

Because A_0 is tridiagonal, this vector is determined by the leading two entries

$$(A_0 - \mu I)e_1 = \begin{bmatrix} d_1 - \mu \\ e_1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

The implicit Francis step starts from exactly the same data: its first Givens rotation is chosen from $(d_1 - \mu, e_1)$, so the orthogonal transformation U generated by bulge chasing has the same first column as Q_e , up to sign. Equivalently, the first column supplied to the theorem is the same whether one starts from the explicit QR factorization or from the implicit bulge-chasing construction. Thus the theorem applies directly to $H = A_0 - \mu I$: since both Q_e and U reduce the same unreduced Hessenberg matrix while sharing this same first-column vector, they must agree up to signs. Therefore the implicit bulge-chasing transform is algebraically the same as the explicit shifted QR step,

$$A_0 - \mu I = Q_e R_e, \quad A_1 = \mu I + R_e Q_e = Q_e^T A_0 Q_e.$$

This is the key reason implicit shifted QR is both structure-preserving and mathematically equivalent to explicit shifted QR [9, 8].

Core Derivation of the Implicit-Shift QR Step

The central algebraic claim is that one implicit Francis sweep produces exactly the same updated matrix as one explicit shifted QR step. We now derive this claim in four steps.

Step 1: Start from the explicit shifted QR factorization. Let

$$A_0 - \mu I = QR,$$

where Q is orthogonal and R is upper triangular. The corresponding explicit shifted QR update is defined by

$$A_1 := \mu I + RQ.$$

Using $R = Q^T(A_0 - \mu I)$, we obtain

$$A_1 = \mu I + Q^T(A_0 - \mu I)Q \tag{2.2.4}$$

$$= Q^T A_0 Q. \tag{2.2.5}$$

Therefore one explicit shifted QR step is an orthogonal similarity transformation.

Step 2: Determine the first column of Q . Since A_0 is tridiagonal,

$$(A_0 - \mu I)e_1 = \begin{bmatrix} d_1 - \mu \\ e_1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \alpha q_1, \quad \alpha = \|(A_0 - \mu I)e_1\|_2.$$

Thus the shift determines the first column q_1 of the orthogonal factor.

Step 3: Construct the same first column implicitly. Choose a Givens rotation G_1 such that

$$G_1^T q_1 = e_1.$$

Since $q_1 = \pm(A_0 - \mu I)e_1 / \|(A_0 - \mu I)e_1\|_2$ has nonzero entries only in its first two positions, G_1 acts only in the $(1, 2)$ -plane. Hence

$$G_1^T \frac{(A_0 - \mu I)e_1}{\|(A_0 - \mu I)e_1\|_2} = \pm e_1,$$

so, after multiplying by G_1 , we obtain

$$G_1 e_1 = \pm \frac{(A_0 - \mu I)e_1}{\|(A_0 - \mu I)e_1\|_2} = \pm q_1.$$

Thus the first column of G_1 is already the same as the first column of the explicit QR factor, up to sign. Then

$$A_0^{(1)} = G_1^T A_0 G_1$$

creates a single fill-in at $(3, 1)$, the bulge. Continue with Givens rotations $G_2, \dots, G_{p-1}$ (for an active block of order p) that successively remove the current bulge and create the next one one position lower. After the last rotation,

$$A_1^* = U^T A_0 U, \quad U = G_1 G_2 \cdots G_{p-1},$$

and A_1^* is again tridiagonal. This follows by repeated substitution:

$$A_0^{(2)} = G_2^T A_0^{(1)} G_2 = G_2^T G_1^T A_0 G_1 G_2,$$

and continuing in the same way gives

$$A_0^{(p-1)} = G_{p-1}^T \cdots G_2^T G_1^T A_0 G_1 G_2 \cdots G_{p-1} = U^T A_0 U.$$

Moreover, for $k \geq 2$, the rotation G_k acts on indices $(k, k+1)$ and therefore leaves e_1 unchanged: $G_k e_1 = e_1$. Hence $G_2, \dots, G_{p-1}$ do not affect e_1 , so the product applied to e_1 reduces to the action of the first rotation alone. Consequently,

$$U e_1 = G_1 G_2 \cdots G_{p-1} e_1 = G_1 e_1 = \pm q_1,$$

so the full bulge-chasing transform U has the same first column as G_1 , and hence as the explicit QR factor Q , up to sign.

Step 4: Invoke the Implicit Q theorem. Because $q_1 \propto (A_0 - \mu I)e_1$, the first column of the orthogonal factor is fixed. Under the unreduced Hessenberg assumptions, the Implicit Q theorem implies that the orthogonal factor is then uniquely determined up to signs. Hence the bulge-chasing orthogonal factor U agrees with the explicit QR factor Q up to signs, and therefore

$$A_1^* = U^T A_0 U = Q^T A_0 Q = A_1.$$

Thus one implicit Francis sweep is exactly one explicit shifted QR step, but carried out in a form that preserves tridiagonal structure.

2.2.9 One Francis Sweep (Algorithmic Form)

One implicit shifted QR sweep on an active block $T_{[l:m, l:m]}$ follows:

1. Choose shift μ (typically Wilkinson from the trailing 2×2 block).
2. Form the first Givens from $(a_l - \mu, b_l)$ and create the bulge.
3. Chase the bulge using adjacent Givens rotations down to the block end.
4. Update the tridiagonal entries (a, b) and, if needed, apply the same rotations to eigenvector accumulator Z .
5. Test deflation criteria; set small subdiagonals to zero and split blocks.

This sweep is the core kernel used by **STEQR**. The complete **STEQR** iteration is obtained by repeating this kernel on the current active unreduced tridiagonal block: choose the block, compute the shift, perform one implicit Francis sweep, check whether the trailing subdiagonal has become small enough to deflate, then either split the block or continue on the remaining active part [9, 10, 2].

2.2.10 Worked Example on a Common Test Matrix

Consider the following 6×6 symmetric tridiagonal test matrix:

$$T = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 2 & 1 & 0 & 0 & 0 \\ 0 & 1 & 3 & 1 & 0 & 0 \\ 0 & 0 & 1 & 4 & 1 & 0 \\ 0 & 0 & 0 & 1 & 5 & 1 \\ 0 & 0 & 0 & 0 & 1 & 6 \end{bmatrix}.$$

This example instantiates the preceding theory on a concrete symmetric tridiagonal matrix. The shift is chosen from the trailing 2×2 block by the Wilkinson rule, the first Givens rotation is determined by $(T - \mu I)e_1$, and the resulting bulge-chasing process realizes one implicit shifted QR sweep. By the derivation above, this sweep is algebraically equivalent to one explicit shifted QR step while preserving tridiagonal structure. In **STEQR**, the matrix would not be stored as a full 6×6 array during this process; instead, the sweep updates only the diagonal and subdiagonal data of the current active unreduced block.

At the start, the full matrix is the active block, so the current unreduced block has order $m = 6$. One implicit shifted sweep consists of choosing a Wilkinson shift from the trailing 2×2 principal submatrix, creating a bulge from the first shifted column, chasing that bulge to the bottom, and then testing whether the trailing subdiagonal has become small enough to deflate.

Its trailing 2×2 block is

$$\begin{bmatrix} 5 & 1 \\ 1 & 6 \end{bmatrix},$$

so the Wilkinson shift is

$$\mu \approx 6.6180.$$

Following the derivation above, form the first shifted two-vector

$$x = \begin{bmatrix} d_1 - \mu \\ e_1 \end{bmatrix} = \begin{bmatrix} 1 - 6.6180 \\ 1 \end{bmatrix} \approx \begin{bmatrix} -5.6180 \\ 1 \end{bmatrix}.$$

The first rotation is a 6×6 Givens rotation acting on indices $(1, 2)$:

$$G_1 = \begin{bmatrix} c_1 & -s_1 & 0 & 0 & 0 & 0 \\ s_1 & c_1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad c_1^2 + s_1^2 = 1.$$

Its active 2×2 block is therefore

$$\begin{bmatrix} c_1 & -s_1 \\ s_1 & c_1 \end{bmatrix}.$$

The parameters are chosen so that this active block satisfies

$$\begin{bmatrix} c_1 & s_1 \\ -s_1 & c_1 \end{bmatrix} \begin{bmatrix} d_1 - \mu \\ e_1 \end{bmatrix} = \begin{bmatrix} r \\ 0 \end{bmatrix},$$

This means that c_1 and s_1 are selected to annihilate the second entry of the shifted two-vector while leaving the remaining coordinates unchanged in the embedded 6×6 rotation. Equivalently,

$$r = \sqrt{(d_1 - \mu)^2 + e_1^2}, \quad c_1 = \frac{d_1 - \mu}{r}, \quad s_1 = \frac{e_1}{r},$$

with the sign convention used here. The numerical values are

$$r \approx 5.7063, \quad c_1 \approx -0.9845, \quad s_1 \approx 0.1752.$$

The first similarity

$$T^{(1)} = G_1^T T G_1$$

creates the bulge at $(3, 1)$. Numerically,

$$T^{(1)} \approx \begin{bmatrix} 0.6856 & 0.7660 & 0.1752 & 0 & 0 & 0 \\ 0.7660 & 2.3144 & -0.9845 & 0 & 0 & 0 \\ 0.1752 & -0.9845 & 3 & 1 & 0 & 0 \\ 0 & 0 & 1 & 4 & 1 & 0 \\ 0 & 0 & 0 & 1 & 5 & 1 \\ 0 & 0 & 0 & 0 & 1 & 6 \end{bmatrix},$$

so the out-of-band entry at $(3, 1)$ is directly visible. To isolate the entries modified in this first local update, the leading 3×3 window changes from

$$\begin{bmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 3 \end{bmatrix} \quad \text{to} \quad \begin{bmatrix} \mathbf{0.6856} & \mathbf{0.7660} & \mathbf{0.1752} \\ \mathbf{0.7660} & \mathbf{2.3144} & \mathbf{-0.9845} \\ \mathbf{0.1752} & \mathbf{-0.9845} & \mathbf{3} \end{bmatrix},$$

so the new fill-in appears precisely at the symmetric pair $(1, 3)$ and $(3, 1)$.

To make the chase itself visible, apply the next Givens rotation G_2 on indices $(2, 3)$. This removes the bulge at $(3, 1)$ and creates the next bulge at $(4, 2)$. Numerically,

$$T^{(2)} \approx \begin{bmatrix} 0.6856 & 0.7858 & 0 & 0 & 0 & 0 \\ 0.7858 & 1.9204 & -0.7376 & 0.2230 & 0 & 0 \\ 0 & -0.7376 & 3.3939 & 0.9748 & 0 & 0 \\ 0 & 0.2230 & 0.9748 & 4 & 1 & 0 \\ 0 & 0 & 0 & 1 & 5 & 1 \\ 0 & 0 & 0 & 0 & 1 & 6 \end{bmatrix},$$

so the bulge has moved from $(3, 1)$ to $(4, 2)$. The later rotations are constructed in exactly the same way: at each stage, G_k is built from the two entries containing the current bulge, and it is chosen so that the lower entry is annihilated. This removes the present bulge and creates the next one one position farther down. Equivalently, the affected leading 4×4 window changes from

$$\begin{bmatrix} 0.6856 & 0.7660 & 0.1752 & 0 \\ 0.7660 & 2.3144 & -0.9845 & 0 \\ 0.1752 & -0.9845 & 3 & 1 \\ 0 & 0 & 1 & 4 \end{bmatrix} \quad \text{to} \quad \begin{bmatrix} 0.6856 & \mathbf{0.7858} & \mathbf{0} & \mathbf{0} \\ \mathbf{0.7858} & \mathbf{1.9204} & \mathbf{-0.7376} & \mathbf{0.2230} \\ \mathbf{0} & \mathbf{-0.7376} & \mathbf{3.3939} & \mathbf{0.9748} \\ 0 & \mathbf{0.2230} & \mathbf{0.9748} & 4 \end{bmatrix},$$

so the old bulge at $(3, 1)$ is eliminated, while the new symmetric fill-in appears at $(2, 4)$ and $(4, 2)$.

The same mechanism repeats: G_3 moves the bulge to $(5, 3)$, G_4 moves it to $(6, 4)$, and G_5 removes the last bulge so that it exits the active block. Hence the numerical sequence itself already shows the bulge-chasing process without needing a separate schematic. This is the essential idea of implicit shifted QR: the shift is encoded through the first local Givens rotation, and the remaining orthogonal transformation is realized by bulge chasing, so the method produces the shifted QR update without ever forming a dense factorization of $T - \mu I$.

The numerical data for the first sweep are summarized in Table 2.1.

Step	indices	input vector	(c_k, s_k)	bulge after step
G_1	$(1, 2)$	$(-5.6180, 1)$	$(-0.9845, 0.1752)$	$(3, 1)$
G_2	$(2, 3)$	$(0.7660, 0.1752)$	$(0.9748, 0.2230)$	$(4, 2)$
G_3	$(3, 4)$	$(-0.7376, 0.2230)$	$(-0.9572, 0.2894)$	$(5, 3)$
G_4	$(4, 5)$	$(0.6436, 0.2894)$	$(0.9120, 0.4101)$	$(6, 4)$
G_5	$(5, 6)$	$(-0.4442, 0.4101)$	$(-0.7347, 0.6783)$	exits

Table 2.1: Numerical data for one implicit Francis sweep on the 6×6 test matrix.

Thus the bulge path is

$$(3, 1) \rightarrow (4, 2) \rightarrow (5, 3) \rightarrow (6, 4) \rightarrow \text{exit}.$$

After the final rotation, tridiagonality is restored:

$$T^+ = (G_1 G_2 G_3 G_4 G_5)^T T (G_1 G_2 G_3 G_4 G_5).$$

The first sweep changes the tridiagonal data from

$$d = [1, 2, 3, 4, 5, 6], \quad e = [1, 1, 1, 1, 1]$$

to

$$d^+ \approx [0.6856, 1.9204, 2.9046, 3.8592, 4.8912, 6.7390],$$

$$e^+ \approx [0.7858, 0.7705, 0.7057, 0.6046, -0.1116].$$

In particular, the trailing subdiagonal is reduced from

$$|e_5| = 1 \quad \text{to} \quad |e_5^+| \approx 1.1164 \times 10^{-1},$$

which is the quantitative signal that the sweep is pushing the active block toward deflation.

For this test matrix, repeated Wilkinson-shift sweeps reduce the subdiagonal entries rapidly. A short summary is

sweep	0	1	2	3
$ e_5 $	1	1.1164×10^{-1}	3.4294×10^{-5}	5.0295×10^{-16}

so the trailing entry deflates after only a few sweeps. In the practical STEQR loop, once $|e_5|$ falls below the deflation threshold, the 6×6 active block splits into a 5×5 leading block and a 1×1 trailing block, and the process continues on the remaining active block. This example therefore shows the complete logic of STEQR in miniature: choose a shift, create a bulge, chase it, restore tridiagonality, test the trailing subdiagonal, and deflate.

If eigenvectors are requested, the same rotations are accumulated sweep by sweep. After one sweep on the active block, the local eigenvector accumulator is

$$Z^{(1)} = G_1 G_2 G_3 G_4 G_5 =: U,$$

so that

$$T^+ = (Z^{(1)})^T T Z^{(1)}.$$

Here $Z^{(1)}$ is exactly the accumulated orthogonal transformation produced by the implicit sweep. In the notation of the derivation above, this product is the bulge-chasing orthogonal factor U . By the Implicit Q theorem, U agrees, up to signs, with the orthogonal factor Q in the explicit shifted QR factorization

$$T - \mu I = QR.$$

After repeated sweeps and deflations, the full iteration produces

$$T^{(k)} \rightarrow \text{diag}(\lambda_1, \dots, \lambda_6), \quad Z^{(k)} \rightarrow [z_1, \dots, z_6],$$

where the columns z_i are the orthonormal eigenvectors of the tridiagonal matrix. Thus the practical output of STEQR is a diagonal matrix of converged eigenvalues together with, when requested, the accumulated eigenvector matrix.

With shifts, eigenvalues may converge and deflate in a non-monotone order, so they are not generally obtained in sorted order during the iteration. For this matrix, the eigenvalues are reported below in ascending order:

$$\lambda \approx [0.2538, 1.7895, 2.9649, 4.0351, 5.2105, 6.7462].$$

To illustrate the eigenvector output without listing the entire 6×6 orthogonal matrix, one may show one representative rounded column. For the smallest eigenvalue $\lambda_1 \approx 0.2538$, the associated normalized eigenvector is approximately

$$z_1 \approx \begin{bmatrix} 0.7800 \\ -0.5800 \\ 0.2400 \\ -0.0700 \\ 0.0100 \\ 0 \end{bmatrix}.$$

Hence the example exhibits both outputs of **STEQR**: the converged eigenvalues and, when eigenvectors are requested, columns of the accumulated orthogonal matrix.

2.2.11 Practical Remarks

One Francis sweep should be distinguished from the full **STEQR** iteration. A single sweep applies one implicit shifted QR step to the current active unreduced block. The complete solver then repeats the same sweep–deflation loop until all active blocks have become diagonal.

If only eigenvalues are required, the iteration updates only the tridiagonal data (a, b) . If eigenvectors are also required, the same Givens rotations are accumulated into an orthogonal matrix Z , so that the final dense eigenvectors are obtained as $Q_{\text{trd}}Z$ after the initial tridiagonalization. This accumulation preserves orthogonality but also makes the full-eigenvector path more sequential.

For a symmetric tridiagonal matrix, one implicit-shift sweep costs $O(n)$, so the eigenvalue-only path costs $O(n^2)$ over all sweeps. The method is backward stable provided the implementation uses robust Givens constructions, a cancellation-safe Wilkinson formula, and machine-scaled deflation tests [2, 7].

Implicit QR/QL on a tridiagonal matrix is therefore robust and memory-light, but less attractive for large full-eigenvector problems because of its long sequential rotation chains. This limitation motivates divide-and-conquer (**STEDC**), developed in Section 2.3.

2.3 Divide-and-Conquer Method for Tridiagonal Eigenproblems

2.3.1 Motivation and Roadmap

Classical QR/QL iteration, as discussed in Section 2.2, is robust and mathematically elegant, but on the full eigenvector path it is also sequential and expensive. A standard back-of-the-envelope estimate is about $6n^3$ flops: each implicit-shift sweep applies about n Givens rotations to an $n \times n$ eigenvector matrix, and about n sweeps are needed in total [2, 7]. This structure offers limited opportunity for concurrency because each sweep depends on the previous one.

The divide-and-conquer (D&C) strategy addresses the same tridiagonal eigenproblem from a different angle. Instead of evolving the matrix through many local QR sweeps, it first splits the tridiagonal matrix into smaller independent subproblems and then merges their eigensystems through a rank-one correction. In this form, the dominant work is reorganized into larger matrix operations, which is why the method becomes attractive when both eigenvalues and eigenvectors are computed [3, 4].

This section follows a single line of ideas: split the tridiagonal matrix, diagonalize the subproblems, reduce the merge problem to a diagonal-plus-rank-one form, solve the secular equation, reconstruct the eigenvectors, and finally explain why deflation and recursion make the method effective in practice.

Notation: For a symmetric tridiagonal matrix, a_i denotes diagonal entries and b_i subdiagonal entries. Standard basis vectors are still written e_i . The scalar ρ denotes the coupling removed at the splitting point.

2.3.2 Divide Step: Splitting into Compensated Blocks

Consider a real symmetric tridiagonal matrix

$$T = \begin{bmatrix} a_1 & b_1 & & & \\ b_1 & a_2 & b_2 & & \\ & b_2 & \ddots & \ddots & \\ & & \ddots & a_{n-1} & b_{n-1} \\ & & & b_{n-1} & a_n \end{bmatrix} \in \mathbb{R}^{n \times n}.$$

Choose a splitting index $m \approx n/2$ and isolate the coupling $\rho := b_m$. If one first separates the matrix into its two principal blocks, the raw decomposition is

$$T = \text{blkdiag}(\hat{T}_1, \hat{T}_2) + \rho(e_m e_{m+1}^T + e_{m+1} e_m^T),$$

where $\hat{T}_1 = T(1:m, 1:m)$ and $\hat{T}_2 = T(m+1:n, m+1:n)$. This already decouples the two halves structurally, but the coupling term is rank two rather than rank one.

The key algebraic trick is therefore to absorb a diagonal correction into each block. Define the compensated tridiagonal matrices

$$T_1 = \hat{T}_1 - \rho e_m e_m^T, \quad T_2 = \hat{T}_2 - \rho e_1 e_1^T,$$

where $e_m \in \mathbb{R}^m$ and $e_1 \in \mathbb{R}^{n-m}$ are local standard basis vectors. With

$$u = \begin{bmatrix} e_m \\ e_1 \end{bmatrix},$$

the identity

$$uu^T = \begin{bmatrix} e_m e_m^T & e_m e_1^T \\ e_1 e_m^T & e_1 e_1^T \end{bmatrix}$$

shows the following standard reformulation.

Proposition 2.3.1 (Compensated Split and Rank-One Reduction). *Let T be a symmetric tridiagonal matrix and let $\rho = b_m$ be the subdiagonal coupling at the*

chosen splitting index m . With the compensated blocks T_1, T_2 defined above and $u = \begin{bmatrix} e_m \\ e_1 \end{bmatrix}$, one has

$$T = \text{blkdiag}(T_1, T_2) + \rho uu^\top. \quad (2.3.1)$$

If, in addition, $T_1 = Q_1 \Lambda_1 Q_1^\top$ and $T_2 = Q_2 \Lambda_2 Q_2^\top$, then with $Q_0 = \text{blkdiag}(Q_1, Q_2)$ and $v = Q_0^\top u$, the transformed problem satisfies

$$Q_0^\top T Q_0 = (\Lambda_1 \oplus \Lambda_2) + \rho vv^\top.$$

This is the standard Cuppen reformulation, later stabilized and refined by Gu–Eisenstat [3, 4]. The split is useful not merely because it produces two smaller matrices, but because it converts their coupling into a form that can later be analyzed through a scalar equation.

Proof. By construction,

$$\text{blkdiag}(T_1, T_2) = \text{blkdiag}(\widehat{T}_1, \widehat{T}_2) - \rho \text{blkdiag}(e_m e_m^\top, e_1 e_1^\top).$$

Since

$$uu^\top = \begin{bmatrix} e_m e_m^\top & e_m e_1^\top \\ e_1 e_m^\top & e_1 e_1^\top \end{bmatrix},$$

adding $\rho uu^\top$ restores the two interface diagonal terms and introduces the off-diagonal coupling

$$\rho \begin{bmatrix} 0 & e_m e_1^\top \\ e_1 e_m^\top & 0 \end{bmatrix},$$

which is exactly the coupling between the two raw principal blocks at the split location. Hence

$$\text{blkdiag}(T_1, T_2) + \rho uu^\top = \text{blkdiag}(\widehat{T}_1, \widehat{T}_2) + \rho (e_m e_{m+1}^\top + e_{m+1} e_m^\top) = T.$$

For the transformed form, use

$$\text{blkdiag}(T_1, T_2) = Q_0 (\Lambda_1 \oplus \Lambda_2) Q_0^\top$$

with $Q_0 = \text{blkdiag}(Q_1, Q_2)$, and multiply (2.3.1) on the left by $Q_0^\top$ and on the right by Q_0 . This gives

$$Q_0^\top T Q_0 = (\Lambda_1 \oplus \Lambda_2) + \rho (Q_0^\top u)(Q_0^\top u)^\top,$$

that is,

$$Q_0^\top T Q_0 = (\Lambda_1 \oplus \Lambda_2) + \rho vv^\top. \square$$

$$\left[\begin{array}{c|c} \hat{T}_1 & \rho e_m e_1^\top \\ \hline \rho e_1 e_m^\top & \hat{T}_2 \end{array} \right] \Rightarrow \text{blkdiag}(T_1, T_2) + \rho u u^\top, \quad u = \begin{bmatrix} e_m \\ e_1 \end{bmatrix}.$$

Figure 2.1: Schematic split at the interface between rows and columns m and $m + 1$. The raw principal blocks are first separated and then compensated so that the remaining coupling becomes a rank-one correction.

Running Example. To keep the discussion concrete, reuse the 6×6 tridiagonal test matrix from Section 2.2:

$$T = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 2 & 1 & 0 & 0 & 0 \\ 0 & 1 & 3 & 1 & 0 & 0 \\ 0 & 0 & 1 & 4 & 1 & 0 \\ 0 & 0 & 0 & 1 & 5 & 1 \\ 0 & 0 & 0 & 0 & 1 & 6 \end{bmatrix}, \quad m = 3, \quad \rho = 1.$$

The raw principal blocks are

$$\hat{T}_1 = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 3 \end{bmatrix}, \quad \hat{T}_2 = \begin{bmatrix} 4 & 1 & 0 \\ 1 & 5 & 1 \\ 0 & 1 & 6 \end{bmatrix}.$$

After compensation,

$$T_1 = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix}, \quad T_2 = \begin{bmatrix} 3 & 1 & 0 \\ 1 & 5 & 1 \\ 0 & 1 & 6 \end{bmatrix},$$

and

$$u = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}.$$

The example is small enough that the full algebra can be written down explicitly, but it still shows the same split-and-merge mechanism used at larger sizes.

2.3.3 Subproblem Diagonalization and Recursive Structure

Once the compensated split has been formed, the two subproblems are diagonalized independently:

$$T_1 = Q_1 \Lambda_1 Q_1^\top, \quad T_2 = Q_2 \Lambda_2 Q_2^\top, \quad Q_i^\top Q_i = I. \quad (2.3.2)$$

If a subproblem is still too large, the same splitting idea is applied recursively. The divide stage therefore produces a binary tree of smaller tridiagonal eigenproblems, terminated only when the leaf blocks are small enough to be solved directly.

At this stage the subproblems are completely independent. This independence is the first source of parallelism in the D&C method, but it is also important mathematically: once T_1 and T_2 have been diagonalized, all remaining global information about the original matrix must enter through the rank-one coupling in (2.3.1). The merge step is therefore not a generic combination of two eigensystems; it is a highly structured correction to two already diagonal pieces.

Running Example. For the 6×6 example, the eigendecompositions of the two 3×3 blocks yield

$$\Lambda_1 = \text{diag}(0.1981, 1.5550, 3.2470), \quad \Lambda_2 = \text{diag}(2.5395, 4.7609, 6.6996).$$

Only the boundary vectors $Q_1^T e_3$ and $Q_2^T e_1$ are needed for the later merge. Numerically,

$$Q_1^T e_3 \approx \begin{bmatrix} 0.3280 \\ 0.7370 \\ 0.5910 \end{bmatrix}, \quad Q_2^T e_1 \approx \begin{bmatrix} 0.9018 \\ 0.4042 \\ 0.1531 \end{bmatrix}.$$

These six numbers carry the entire influence of the original coupling into the merge stage.

2.3.4 Merge Step: Diagonal-Plus-Rank-One Form

Assemble

$$Q_0 := \text{blkdiag}(Q_1, Q_2), \quad D := \Lambda_1 \oplus \Lambda_2.$$

Applying the block-diagonal orthogonal matrix Q_0 to Equation (2.3.1) gives

$$Q_0^T T Q_0 = Q_0^T \text{blkdiag}(T_1, T_2) Q_0 + \rho (Q_0^T u)(Q_0^T u)^T \quad (2.3.3)$$

$$= \text{blkdiag}(\Lambda_1, \Lambda_2) + \rho v v^T, \quad (2.3.4)$$

where

$$v := Q_0^T u = \begin{bmatrix} Q_1^T e_m \\ Q_2^T e_1 \end{bmatrix}. \quad (2.3.5)$$

Hence

$$Q_0^T T Q_0 = D + \rho v v^T. \quad (2.3.6)$$

This is the key structural reduction of the method: after the split and local diagonalizations, the global merge problem is no longer tridiagonal, but it is only a rank-one modification of a diagonal matrix. The split by itself is not yet enough; what matters is that the remaining interaction has the special form $D + \rho v v^T$, for which the eigenvalues can be recovered from a one-dimensional secular equation.

Equation (2.3.5) also shows why only boundary information from the subproblems is needed. The vectors $Q_1^T e_m$ and $Q_2^T e_1$ are just the last row of Q_1 and the first row of Q_2 , written as column vectors. In other words, after diagonalizing the subproblems, the merge does not need the full matrices T_1 and T_2 again; it only needs their eigenvalues and the way their eigenvectors touch the split interface.

For the secular analysis it is convenient to sort the diagonal entries of D . If P is a permutation matrix such that $P^T D P$ is diagonal with ascending entries, then

$$P^T(D + \rho v v^T)P = (P^T D P) + \rho (P^T v)(P^T v)^T.$$

Hence reordering does not change the structure of the merge problem; it merely relabels the poles and applies the same permutation to the coupling vector.

Running Example. For the 6×6 example,

$$D = \text{diag}(0.1981, 1.5550, 3.2470, 2.5395, 4.7609, 6.6996),$$

and the transformed update vector is

$$v \approx \begin{bmatrix} 0.3280 \\ 0.7370 \\ 0.5910 \\ 0.9018 \\ 0.4042 \\ 0.1531 \end{bmatrix}.$$

At this point, the original tridiagonal problem has been reduced completely to the eigenproblem for $D + \rho v v^T$.

2.3.5 Secular Equation

Consider the eigenproblem

$$(D + \rho v v^T)y = \lambda y.$$

Assume first that $\lambda \neq d_i$ for every diagonal entry d_i of D . Rearranging gives

$$(\lambda I - D)y = \rho v(v^T y).$$

The right-hand side is a scalar multiple of v , so any nonzero eigenvector y must be parallel to $(\lambda I - D)^{-1}v$. Writing $\alpha := \rho(v^T y)$, one obtains

$$y = \alpha(\lambda I - D)^{-1}v.$$

Multiplying by v^T and dividing by the nonzero scalar $v^T y$ yields

$$1 = \rho v^T (\lambda I - D)^{-1} v = \rho \sum_{i=1}^n \frac{v_i^2}{\lambda - d_i}. \quad (2.3.7)$$

Equivalently, the eigenvalues are the roots of the secular equation

$$f(\lambda) := 1 - \rho \sum_{i=1}^n \frac{v_i^2}{\lambda - d_i} = 0. \quad (2.3.8)$$

Thus the $n \times n$ eigenproblem for $D + \rho v v^T$ is reduced to a scalar root-finding problem for a rational function. The next proposition summarizes the standard secular characterization used in divide-and-conquer.

To see the geometric structure of the roots, reorder the diagonal entries if necessary and write

$$\tilde{D} := P^T D P = \text{diag}(\delta_1, \dots, \delta_n), \quad \delta_1 < \delta_2 < \dots < \delta_n.$$

Let $\tilde{v} := P^T v$. Then the secular equation becomes

$$1 - \rho \sum_{i=1}^n \frac{\tilde{v}_i^2}{\lambda - \delta_i} = 0.$$

Proposition 2.3.2 (Secular Equation and Interlacing). *Let $\tilde{D} = \text{diag}(\delta_1, \dots, \delta_n)$ with $\delta_1 < \delta_2 < \dots < \delta_n$, and consider the active nondeflated rank-one modification $\tilde{D} + \rho \tilde{v} \tilde{v}^T$ with $\rho > 0$. Then its eigenvalues are exactly the roots of*

$$1 - \rho \sum_{i=1}^n \frac{\tilde{v}_i^2}{\lambda - \delta_i} = 0.$$

Moreover, these roots strictly interlace the poles:

$$\delta_1 < \lambda_1 < \delta_2 < \lambda_2 < \dots < \delta_n < \lambda_n.$$

This characterization and the associated interlacing behavior are standard for rank-one modifications of diagonal matrices in the divide-and-conquer literature [7, 4]. For the active nondeflated problem with $\rho > 0$, the proof is a direct consequence of the monotonicity of the secular function between consecutive poles. Indeed,

$$f'(\lambda) = \rho \sum_{i=1}^n \frac{\tilde{v}_i^2}{(\lambda - \delta_i)^2} > 0$$

away from the poles, so f is strictly increasing on each interval between two consecutive poles. Moreover,

$$f(\lambda) \rightarrow -\infty \quad \text{as } \lambda \downarrow \delta_i, \quad f(\lambda) \rightarrow +\infty \quad \text{as } \lambda \uparrow \delta_{i+1}.$$

Therefore there is exactly one root in each interval (δ_i, δ_{i+1}) , and one additional root in (δ_n, ∞) . In particular,

$$\delta_1 < \lambda_1 < \delta_2 < \lambda_2 < \dots < \delta_n < \lambda_n.$$

This interlacing property is one of the main reasons the secular equation is numerically manageable.

Running Example. For the 6×6 example, it is convenient to sort the diagonal entries and permute the components of v consistently. If P denotes the associated permutation matrix, then $P^T D P$ and $P^T v$ are

$$\tilde{D} = \text{diag}(0.1981, 1.5550, 2.5395, 3.2470, 4.7609, 6.6996),$$

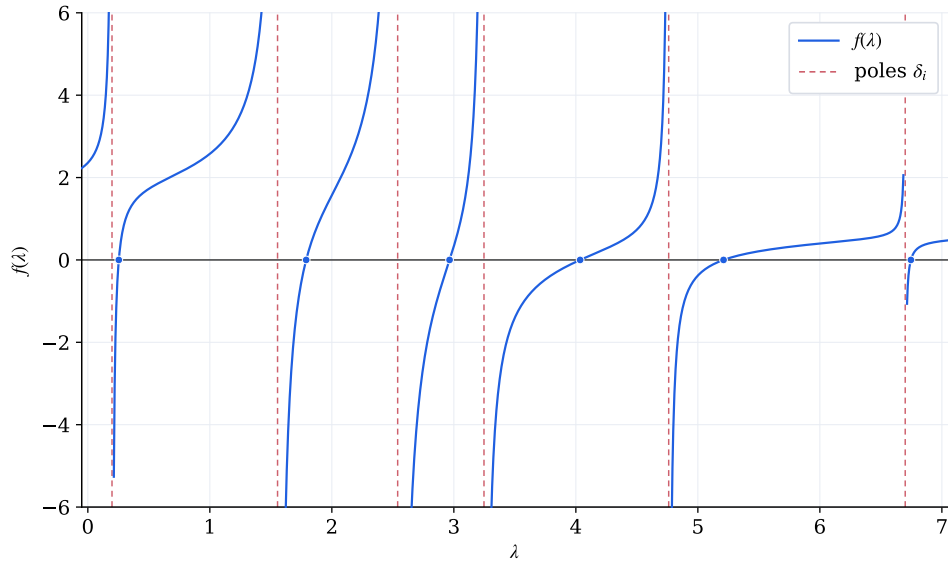


Figure 2.2: Secular function $f(\lambda)$ for the 6×6 running example. The dashed lines mark the poles δ_i , while the blue intersections with the horizontal axis indicate the roots λ_i .

$$\tilde{v} \approx \begin{bmatrix} 0.3280 \\ 0.7370 \\ 0.9018 \\ 0.5910 \\ 0.4042 \\ 0.1531 \end{bmatrix}, \quad \rho = 1.$$

The secular function is therefore approximately

$$f(\lambda) \approx 1 - \frac{0.1076}{\lambda - 0.1981} - \frac{0.5431}{\lambda - 1.5550} - \frac{0.8132}{\lambda - 2.5395} - \frac{0.3493}{\lambda - 3.2470} - \frac{0.1634}{\lambda - 4.7609} - \frac{0.0234}{\lambda - 6.6996}.$$

Solving $f(\lambda) = 0$ on the interlacing intervals gives

$$\lambda \approx [0.2538, 1.7895, 2.9649, 4.0351, 5.2105, 6.7462].$$

Figure 2.2 does not replace the argument above, but it makes the interlacing mechanism visible for the running example. The dashed vertical lines mark the poles δ_i , and the graph is monotone between consecutive poles and crosses the horizontal axis exactly once in each admissible interval. What matters here is not the visual steepness itself, but the fact that $f'(\lambda) > 0$ between consecutive poles, which guarantees monotonicity and hence uniqueness of the root in each interlacing interval.

2.3.6 Eigenvector Reconstruction

Once a root λ_j of the secular equation has been found, the corresponding eigenvector in the diagonal basis is obtained from

$$\tilde{y}_j \propto (\lambda_j I - \tilde{D})^{-1} \tilde{v}.$$

If a permutation P was introduced to order the poles, then the eigenvector in the original merge basis is $y_j = P\tilde{y}_j$. After normalization, the eigenvectors of the original tridiagonal matrix follow by transforming back with Q_0 :

$$z_j = Q_0 P \tilde{y}_j, \quad T z_j = \lambda_j z_j. \quad (2.3.9)$$

Collecting all reconstructed vectors into $\tilde{Y} = [\tilde{y}_1, \dots, \tilde{y}_n]$ and $Z = [z_1, \dots, z_n]$, one obtains the full spectral decomposition

$$T = Z \Lambda Z^T, \quad Z = Q_0 P \tilde{Y}.$$

This reconstruction formula explains why the D&C method is especially attractive when eigenvectors are needed. The expensive work is not a long chain of local rotations, as in the QR route, but a structured sequence of matrix-vector and matrix-matrix operations arising from the recursive merges. At each merge level, the new eigenvectors are assembled from previously computed local eigenvectors and the small amount of coupling data encoded in v .

Running Example. For the smallest eigenvalue $\lambda_1 \approx 0.2538$, the vector $(\lambda_1 I - \tilde{D})^{-1} \tilde{v}$ gives the corresponding eigenvector in the ordered diagonal basis. After normalization, undoing the permutation, and applying the back transformation, one obtains a representative eigenvector of the original matrix:

$$z_1 \approx \begin{bmatrix} 0.7770 \\ -0.5798 \\ 0.2354 \\ -0.0667 \\ 0.0146 \\ -0.0025 \end{bmatrix}.$$

As usual, the sign is not unique: both z_1 and $-z_1$ represent the same eigenspace.

2.3.7 Deflation and Reordering

In practical D&C implementations, the secular equation is usually not solved for all n indices simultaneously. Instead, one first looks for deflation, that is, situations in which part of the problem has already decoupled numerically. Two mechanisms are especially important:

- If a component v_i is zero, or numerically negligible, then the corresponding pole d_i is already an eigenvalue of $D + \rho v v^T$, so that index can be removed from the active secular solve.
- If two diagonal entries d_i and d_j are very close, a local orthogonal rotation can be used to combine them so that one component of v vanishes. This idea lies behind the Gu–Eisenstat stabilization strategy [4].

Deflation and reordering are therefore not secondary implementation tricks. They are part of the mathematical mechanism that turns the naive rank-one merge into a stable and efficient algorithm. Reordering places the poles in the monotone order required by the interlacing analysis, while deflation shrinks the active secular problem and improves robustness in clustered spectra.

For the small 6×6 running example above, no substantial deflation occurs, so the secular equation remains six-dimensional. In larger problems, however, deflation can significantly reduce the work and is one reason why the practical cost is often better than the worst-case recurrence suggests.

2.3.8 Complexity and Parallel Structure

From the complexity point of view, the important case is again the computation of both eigenvalues and eigenvectors. In a simplified asymptotic model for that path, ignoring deflation and lower-order leaf costs, the dominant merge work satisfies

$$C(n) = n^3 + 2C(n/2).$$

The term n^3 reflects the cost of combining eigenvector information at a merge node, while the two recursive terms correspond to the two half-size subproblems. Expanding the recursion tree, at level j there are 2^j subproblems of size $n/2^j$, so the total merge work on that level is

$$2^j \left(\frac{n}{2^j} \right)^3 = \frac{n^3}{4^j}.$$

Summing over all levels gives

$$\sum_{j \geq 0} \frac{n^3}{4^j} < \frac{4}{3}n^3,$$

so in this model

$$C(n) \sim \frac{4}{3}n^3.$$

This should be compared with the rough $6n^3$ estimate for the QR route from Section 2.2. The constants are only approximate, and practical performance depends on deflation and implementation details, but the structural contrast is already clear.

The same recursion also exposes useful parallelism. The two child problems can be solved independently, and the merge stage is dominated by larger dense operations rather than long chains of dependent Givens rotations. This is the structural reason why divide-and-conquer is naturally associated with larger matrix operations. The running 6×6 example is far too small to display this asymptotic behavior, but it exhibits the same split/merge pattern on which the large-scale cost analysis rests.

2.3.9 Summary

The divide-and-conquer method solves the same tridiagonal eigenproblem as the QR route, but it proceeds through a different mathematical decomposition:

1. split the tridiagonal matrix into two compensated subproblems,
2. diagonalize the subproblems independently and recursively if needed,
3. transform the merge problem into $D + \rho vv^T$,

4. solve the secular equation and reconstruct eigenvectors,
5. reduce the active problem further through deflation and reordering.

In contrast with implicit QR/QL iteration, which advances by local implicit-shift sweeps and repeated deflation of active tridiagonal blocks, divide-and-conquer proceeds by recursive splitting and structured rank-one merging. That contrast explains why the QR route is more sequential, while divide-and-conquer is more global and better aligned with larger matrix operations. The next section compares the two mathematical procedures directly.

2.4 Comparison of Algorithmic Procedures

Let

$$T \in \mathbb{R}^{n \times n}, \quad T = T^T$$

be tridiagonal. Sections 2.2 and 2.3 have presented two different ways to compute its eigendecomposition. They target the same spectral factorization, so the issue here is not which problem is being solved, but how the computation is organized.

2.4.1 Conceptual Comparison

In exact arithmetic, both methods aim at the same final object,

$$T = Z \Lambda Z^T,$$

so the mathematical target is identical. The difference lies in how the computation moves from the input tridiagonal matrix to that final spectral factorization.

STEQR proceeds by local implicit iteration. Each active unreduced block is updated through shifted QR/QL sweeps, with Givens rotations chasing a bulge through the tridiagonal structure until small subdiagonal entries deflate. Eigenvectors, when requested, are obtained by accumulating those rotations.

STEDC, by contrast, proceeds by global split-and-merge. The matrix is partitioned into smaller subproblems, each subproblem is solved separately, and the solutions are recombined through a rank-one correction and the secular equation. Eigenvectors are then reconstructed from the merged subproblem data.

The main contrast is summarized in Table 2.2.

Table 2.2: Conceptual comparison between implicit QR/QL iteration and divide-and-conquer.

Aspect	QR/QL iteration	Divide-and-conquer
Primary state	active tridiagonal block	$D + \rho v v^T$
Main nonlinear step	implicit shift + bulge chase	secular equation solve
Orthogonal map	$\prod_k G_k$	$Q_0 P Y$
Stopping signal	$ b_i $ deflation	secular residual/interlacing

The key distinction is that QR/QL iteration evolves the solution through many fine-grained local updates, whereas divide-and-conquer reorganizes the problem into subproblems and structured merge operations.

2.4.2 Complexity and Numerical Stability

From the complexity point of view, the two methods differ most clearly when eigenvectors are required. For eigenvalues only, QR/QL iteration on a tridiagonal matrix is typically $O(n^2)$, since each implicit sweep costs $O(n)$ and many such sweeps may be needed. On the full eigenvector path, the repeated accumulation of plane rotations leads to cubic total work.

For divide-and-conquer, the standard model gives the recurrence

$$C(n) = n^3 + 2C(n/2),$$

so the leading-order cost is

$$C(n) \sim \frac{4}{3}n^3,$$

ignoring reductions from deflation. This estimate explains why divide-and-conquer becomes attractive for large full-eigenvector problems: its dominant work is organized around larger matrix operations and recursive subproblems, rather than long chains of sequential rotations.

The two methods also concentrate numerical difficulty in different places. QR/QL iteration is generally very robust, but its sensitivity is tied to long rotation chains, shift selection, and the deflation tests that decide when a subdiagonal entry is negligible. Divide-and-conquer also preserves symmetry and orthogonality well, but its most delicate step is the secular solve, particularly when poles are tightly clustered and deflation or reordering choices become important.

2.4.3 Conclusion

The two procedures therefore have the same mathematical target but different computational forms. QR/QL iteration advances through repeated local rotations and stays relatively light in workspace. Divide-and-conquer spends extra storage in order to split the problem, merge subproblem spectra, and apply larger orthogonal updates. This difference in mathematical organization is the main distinction between the two tridiagonal eigensolvers.

CHAPTER 3

Implementation-Level Comparison of DSYEV and DSYEVD

This chapter bridges the mathematical algorithms of Chapter 2 and the performance results of Chapter 4. Its purpose is not to reproduce LAPACK documentation, but to explain the implementation choices that matter for understanding the practical difference between DSYEV and DSYEVD.

The focus is therefore on implementation structure, data flow, and the stages where the two drivers genuinely diverge. Section 3.1 gives an overview of the two routines and the common three-stage pipeline. Section 3.2 then examines the key implementation blocks, with emphasis on the shared reduction DSYTRD, the QR-based tridiagonal solver DSTEQR, and the divide-and-conquer solver DSTEDC. Section 3.3 summarizes the main implementation implications and explains why these differences matter for the performance study in Chapter 4.

3.1 Overview of the Driver Structure

This section gives a compact overview of DSYEV and DSYEVD before the chapter turns to their main implementation differences. The aim is not to document every internal subroutine, but to identify the small number of design choices that are most relevant for the performance discussion in Chapter 4.

3.1.1 Interface-Level View

At the user level, the two LAPACK routines answer the same basic question: given a real symmetric dense matrix A , compute its eigenvalues and, optionally, its eigenvectors. In both cases the mathematical output is

$$AU = U\Lambda,$$

with the eigenvalues returned in ascending order and the eigenvectors orthonormal.

At the interface level, the most important points are the following:

- JOBZ determines whether only eigenvalues or both eigenvalues and eigenvectors are returned.
- UPLO specifies whether the upper or lower triangular part of the symmetric matrix is supplied.
- DSYEVD solves the same problem as DSYEV, but uses additional workspace in order to employ the divide-and-conquer algorithm.

The two routines are summarized in Table 3.1.

For the present chapter, the external interfaces are not the main source of interest, since they are very similar in both cases. The significant distinction appears after the driver begins its internal work: DSYEV follows a QR-based tridiagonal path, whereas DSYEVD switches to divide-and-conquer.

Table 3.1: Interface-level summary of DSYEV and DSYEVD.

Routine	Main idea	Relevant interface features
DSYEV	Classical QR-based symmetric eigensolver for dense matrices.	Same input/output goal as DSYEVD; smaller workspace; most relevant parameters here are JOBZ, UPLO, and WORK/LWORK.
DSYEVD	Divide-and-conquer version of the same dense symmetric eigensolver.	Interface is almost the same as DSYEV, but it needs larger workspace, including IWORK/LIWORK; this extra storage usually pays off when all eigenvectors are needed.

3.1.2 Common Driver Pipeline

Both drivers follow the same high-level three-stage pipeline; they differ only in the routines used after the shared reduction stage:

1. **Shared front end:** DSYTRD reduces the dense symmetric matrix A to tridiagonal form.
2. **Main point of divergence:** DSYEV uses DSTEQR, while DSYEVD uses DSTEDC.
3. **Eigenvector recovery:** when $JOBZ='V'$, DSYEV forms Q explicitly through DORGTR, whereas DSYEVD applies the stored reflectors later through DORMTR.

The important message is simple: the shared reduction stage does not explain most of the performance gap. The real difference appears in the tridiagonal eigensolver and in how the tridiagonal eigenvectors are mapped back to the dense problem. A compact summary of the main LAPACK routines referenced throughout this chapter is collected in Appendix A.

3.1.3 Main Points of Comparison

The two flowcharts later in this chapter are intended as conceptual workflow diagrams rather than complete LAPACK call graphs. The main contrasts are:

- DSYEV spends most of its tridiagonal-eigenvector work inside a QR-style iteration with many fine-grained updates.
- DSYEVD reorganizes the same task into recursive subproblems, merge steps, and larger matrix operations.
- DSYEVD needs more temporary storage, but this is exactly what allows it to exploit blocked operations and modern BLAS implementations more effectively.

3.2 Key Implementation Blocks

3.2.1 SYTRD: Householder Tridiagonalization in LAPACK

The LAPACK routine DSYTRD reduces a dense symmetric matrix to

$$Q^T A Q = T,$$

where T is tridiagonal. This step is shared by both `DSYEV` and `DSYEVD`, so its main role in this chapter is to provide common implementation context before the two drivers diverge at the tridiagonal eigensolver stage.

In LAPACK, the reduction is carried out with Householder reflectors stored implicitly. The tridiagonal entries are written to the arrays `D` and `E`, while the reflector data are packed into `A` and `TAU`. This design avoids forming the dense orthogonal matrix Q immediately, allowing later routines such as `DORGTR` or `DORMTR` to reconstruct or apply it only when eigenvectors are required.

From the implementation perspective, the key point is that `DSYTRD` uses a blocked structure with two distinct performance regimes:

- **Panel processing (unblocked path):** Columns in the active panel are reduced one-by-one using Householder reflectors. This stage is carried out by routines such as `dlatrd` and `dsytd2`, which internally call Level-1/2 BLAS kernels (e.g., `dlarfg`, `daxpy`, `dsymv`, `dscal`) to construct and apply individual reflectors. This phase is memory-bound and typically limits parallel scalability.
- **Trailing matrix update (blocked path):** Once a block of reflectors has been formed, they are applied together to the unreduced trailing submatrix using the Level 3 BLAS kernel `dsyr2k`. This step achieves high arithmetic intensity and forms the performance-critical portion of the routine.

This split explains why `DSYTRD` is important but not decisive for the `DSYEV` versus `DSYEVD` comparison. Its total cost is still about $\frac{4}{3}n^3$ flops, and the panel phase remains limited by Level 2 BLAS behavior, so both drivers inherit a similar front-end cost. The main source of their later performance difference therefore lies not in `DSYTRD` itself, but in how the tridiagonal problem and eigenvectors are handled after this shared reduction.

3.2.2 *STEQR: Implicit QR/QL Iteration for Tridiagonal Matrices*

Overview and Function of DSTEQR

The LAPACK routine `DSTEQR` implements the implicit QR/QL method for computing all eigenvalues and optionally eigenvectors of a real symmetric *tridiagonal* matrix. In the dense symmetric driver `DSYEV`, it is used after reduction to tridiagonal form:

$$A \xrightarrow{\text{DSYTRD}} Q^\top A Q = T \xrightarrow{\text{DORGTR}} Q \xrightarrow{\text{DSTEQR('V')} \text{ with } Z=Q} U = Q U_T.$$

The implementation point that matters here is that `DSYEV` forms the dense basis Q first and then asks `DSTEQR('V')` to accumulate the tridiagonal rotations directly onto that basis.

Workflow in the DSYEV Path

For the present comparison, the essential point is that the `DSYEV` eigenvector path is built around an iterative QR/QL process on the tridiagonal matrix. Figure 3.1 summarizes this path at a conceptual level.

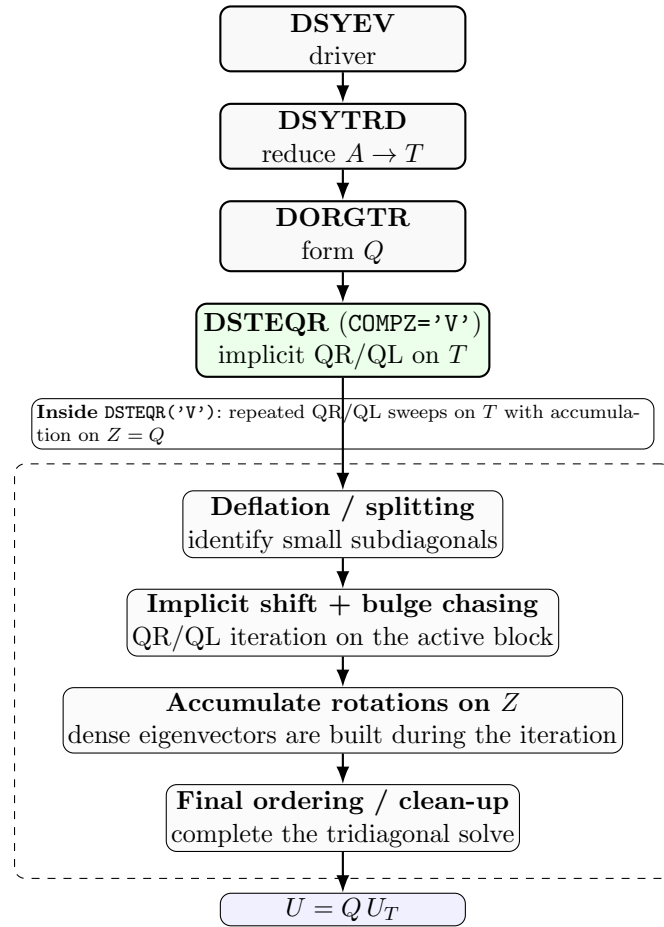


Figure 3.1: Conceptual workflow of the eigenvector path in DSYEV. The important feature is that eigenvector formation stays inside a QR/QL iteration built from many sequential updates.

Figure 3.1 emphasizes the main structural point: DSTEQR repeatedly updates a small active tridiagonal block and applies the same rotations to the dense eigenvector basis. The eigenvector path is therefore dominated by fine-grained sequential work rather than large blocked matrix operations.

Discussion: Eigenvector Path and Practical Implications

Compared to divide-and-conquer, DSTEQR has modest workspace requirements and a regular memory-access pattern. Its accumulation strategy, which applies QR/QL rotations directly to Z , also maintains orthogonality in a straightforward way. However, when many eigenvectors are required for large dense problems, the limited exposure to Level 3 BLAS usually leads to weaker performance and less parallel scalability than DSTEDC. The QR path is therefore best understood as the lighter-weight realization rather than the performance-oriented one.

3.2.3 STEDC: Divide-and-Conquer Implementation in LAPACK

Overview and Function of DSTEDC

The LAPACK routine DSTEDC implements the divide-and-conquer (D&C) method for computing all eigenvalues and eigenvectors of a real symmetric tridiagonal

matrix. In the dense symmetric driver `DSYEVD` with `JOBZ='V'`, LAPACK uses `DSTEDC('I')` to compute the eigenvectors U_T of the tridiagonal matrix T , and then applies the Householder reflectors by `DORMTR` to obtain the eigenvectors of the original dense matrix A . Although `DSTEDC` also supports `COMPZ='V'` in general, that is not the path used by `DSYEVD`.

In the driver routine `DSYEVD`, this step occurs after tridiagonalization:

$$A \xrightarrow{\text{SYTRD}} Q^T A Q = T \xrightarrow{\text{DSTEDC('I')}} U_T \xrightarrow{\text{DORMTR}} U = Q U_T.$$

The key implementation contrast with `DSYEV` is the ordering of work: `DSYEVD` first solves the tridiagonal problem in its own basis and only afterwards maps those vectors back to the dense basis through `DORMTR`.

Workflow in the `DSYEVD` Path

Within the `DSYEVD` driver path, the tridiagonal eigensolver is no longer an extended QR-style iteration. Instead, `DSTEDC('I')` works by splitting the tridiagonal problem into smaller subproblems, solving them, and merging them back together. Figure 3.2 summarizes this workflow at the level needed for the present thesis.

Figure 3.2 again emphasizes structure rather than a complete list of helper routines. What matters is that `DSTEDC` pushes more of the heavy work into split, merge, and blocked-update stages. One implementation detail is worth making explicit. The presence of `DSTEQR` inside `DSTEDC` does not mean that `DSYEVD` falls back to the same cost structure as `DSYEV`. In the D&C path, `DSTEQR` is used only on small leaf subproblems after recursive splitting, whereas `DSYEV` applies `DSTEQR('V')` to the full tridiagonal problem while accumulating eigenvectors in a dense matrix. The routine name therefore overlaps, but its scale and role are different.

Discussion: Eigenvector Path and Performance Implications

Compared with the traditional QR iteration (`DSTEQR`), the D&C approach reduces both arithmetic complexity and bandwidth pressure in the eigenvector path. Inside `DSTEDC`, the dominant work is organized as blocked merge updates, which are well matched to Level-3 BLAS. The final dense-basis mapping is then carried out by `DORMTR`. Although `DSTEDC` requires additional workspace for temporary eigenvector blocks, this cost is compensated by reduced arithmetic and improved cache reuse, which usually yields better scalability for large dense matrices. In practice, the expensive merge path in LAPACK is funneled largely into DGEMM-dominated blocked work, which is exactly the implementation feature that later explains the performance advantage of `DSYEVD`.

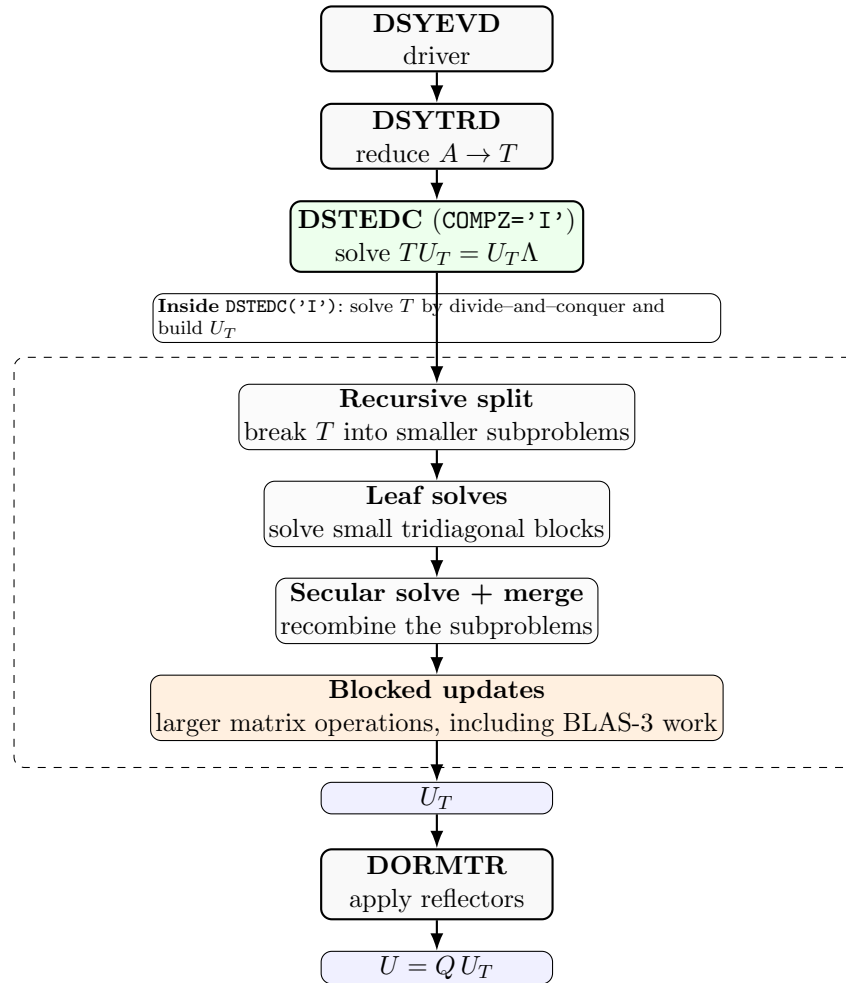


Figure 3.2: Conceptual workflow of the eigenvector path in DSYEVD. The important feature is that the tridiagonal problem is reorganized into recursive subproblems and blocked merge work before the final dense-basis mapping.

3.3 Implementation Implications

Chapter 2 presented the mathematical logic of the symmetric eigenvalue problem in three stages: reduction to tridiagonal form, solution of the tridiagonal eigenproblem, and recovery of eigenvectors for the original dense matrix. The LAPACK implementations preserve this broad structure, but the practical difference between DSYEV and DSYEVD appears in the organization of the second and third stages. The purpose of this section is to distill the implementation details discussed earlier into the specific structural differences that matter for later performance interpretation.

Table 3.2: Implementation-level comparison of the main stages in DSYEV and DSYEVD.

Stage	Driver realization	Why it matters
Dense $\rightarrow$ tridiagonal reduction	DSYTRD with Householder reflectors stored implicitly in A and TAU	Shared by both drivers; blocked updates expose some BLAS-3, but panel work remains memory-bound.
Tridiagonal eigensolution (QR path)	DSTEQR with deflation, implicit shifts, Givens rotations, and optional accumulation on Z	Small workspace and robust structure, but limited Level-3 BLAS usage reduces scalability for large problems.
Tridiagonal eigensolution (D&C path)	DSTEDC through the DLAED* family: splitting, secular solving, deflation, and merging	Requires more workspace, but blocked merge steps map well to DGEMM and modern hardware.
Back transformation to dense eigenvectors	DSYEV: DORGTR + DSTEQR('V'); DSYEVD: DORMTR after DSTEDC('I')	Same mathematical target $U = QU_T$, but different routine ordering and data flow.

Table 3.2 highlights the central implementation point of this chapter. The decisive difference between DSYEV and DSYEVD does not arise in the shared reduction routine DSYTRD; rather, it arises in the tridiagonal eigensolver and in the subsequent treatment of eigenvectors. The DSYEV path, based on DSTEQR, is dominated by sequential bulge-chasing updates and plane rotations. By contrast, the DSYEVD path, based on DSTEDC, reorganizes the same stage into recursive subproblems and blocked merge operations. This reorganization increases workspace requirements, but it also creates substantially more Level-3 BLAS work and correspondingly greater opportunity for cache reuse and parallel execution. From an implementation perspective, DSYEV is therefore the lighter-weight driver, whereas DSYEVD trades additional workspace for a more hardware-efficient structure. This is the bridge to Chapter 4: the empirical advantage of DSYEVD should be sought primarily in the tridiagonal eigensolver and eigenvector-handling stages, not in the shared dense tridiagonalization stage.

CHAPTER 4

Experimental Evaluation and Performance Analysis

This chapter studies the practical difference between `DSYEV` and `DSYEVD` on the test platform, with emphasis on the full-eigenpair case `JOBZ='V'`. It asks three empirical questions. First, which driver is faster end to end? Second, which stage creates most of the observed gap? Third, how much of that gap should be attributed to the tridiagonal algorithm itself, to the optimized BLAS/LAPACK implementation, and to wider SIMD support?

The argument moves from broad timing evidence to narrower attribution. Section 4.1 records the platform, libraries, matrices, and measurement protocol. Section 4.2 establishes the end-to-end ordering and the stage-level breakdown. Sections 4.3 and 4.4 then hold parts of the software stack fixed in order to separate the effect of algorithmic path from the effect of implementation quality and vector width. Section 4.5 adds call-path profiling and hardware-counter evidence, and Section 4.6 draws the chapter-level conclusion.

The short version of that conclusion is that, on this platform, `DSYEVD` gains its advantage mainly by replacing the QR-based tridiagonal eigensolver with `DSTEDC`. The price of that faster path is substantially higher workspace demand.

4.1 Experimental Setup

All experiments were conducted on an AWS EC2 `c7g.xlarge` instance running Linux and equipped with an AWS Graviton3 ARM processor. The following sections summarize only the experimental context needed to read the benchmark results. Additional configuration details are collected in Appendix B.

4.1.1 Hardware Platform

Experiments were performed on an AWS EC2 `c7g.xlarge` instance equipped with an AWS Graviton3 processor (ARMv8-A with SVE support). This distinction matters for the controlled comparison in Section 4.4: the processor supports both the conventional 128-bit ARM SIMD extension (ASIMD/NEON) and the newer SVE vector extension. On this system, Linux reports an SVE default vector length of 32 bytes, that is, 256 bits. The system characteristics are:

- **CPU:** 4 vCPUs, single hardware thread per core
- **Memory:** 8 GiB RAM

All controlled comparisons were run in single-threaded mode to isolate algorithmic and library-level performance differences. The overall benchmark in Section 4.2 additionally includes a small multithread scaling check with 1, 2, and 4 threads on the same machine.

4.1.2 Software Configuration

LAPACK and BLAS Libraries. The experiments use three library configurations, based on Netlib LAPACK and reference BLAS [2, 12, 13] and on OpenBLAS [14]:

Table 4.1: Library configurations for the current DSYEV/DSYEVD experiments.

Configuration	LAPACK layer	BLAS implementation
LAPACK	Reference LAPACK	Reference BLAS
OpenBLAS SIMD	Reference LAPACK	OpenBLAS (SIMD)
OpenBLAS SVE	Reference LAPACK	OpenBLAS (SVE)

The `OpenBLAS SIMD` configuration is built for the generic ARMV8 target without SVE and still uses 128-bit ASIMD/NEON instructions in kernels such as `DGEMM`. The `OpenBLAS SVE` configuration is built for the Graviton3 SVE-capable target (`NEOVERSEV1`). For the benchmark runs in Section 4.2, the active OpenBLAS SVE library is a pthread build with `MAX_THREADS=8`; since the test machine has 4 vCPUs, the thread scaling check is limited to 1, 2, and 4 threads.

For the DSYEV/DSYEVD experiments, all three configurations follow the same Reference LAPACK lineage at the LAPACK layer. In particular, the OpenBLAS SIMD and OpenBLAS SVE builds use the same LAPACK-side routines for DSYEV, DSYEVD, DSYTRD, DSTEQR, DSTEDC, DORGTR, and DORMTR; they differ in the underlying BLAS kernels. OpenBLAS also includes additional integration code and some optimized replacements for other routines, but those are not material to the present DSYEV/DSYEVD study. Exact compiler flags, build targets, and thread-control settings are listed in Appendix B.

Performance Measurement. Timing measurements were obtained using wrapper functions that instrument LAPACK routines at the subroutine level. This allows us to break down the total runtime into:

- Tridiagonal reduction: DSYTRD
- Tridiagonal eigensolver: DSTEQR (for DSYEV) or DSTEDC (for DSYEVD)
- Eigenvector back-transformation: DORGTR (for DSYEV) or DORMTR (for DSYEVD)

The benchmark and controlled-comparison tables in Sections 4.2–4.4 use the same matrix sizes, the same stage-level wrapper scheme, and repeated runs under a fixed threading configuration. The deeper profiling and hardware-counter experiments use heavier instrumentation and are therefore interpreted primarily as explanatory evidence rather than as exact timing replicas. The full run protocol, thread-control settings, and correctness checks are summarized in Appendix B. For reproducibility, the accompanying repository [15] provides the scripts, code, and result snapshots corresponding to the experiments reported in this chapter.

4.1.3 Test Matrices

All experiments use Symmetric Positive Definite (SPD) Kac–Murdock–Szegő (KMS) matrices with correlation parameter $\rho = 0.95$ [16]. A KMS matrix has entries

$$A_{ij} = \rho^{|i-j|}, \quad 0 < \rho < 1,$$

so it is a dense symmetric Toeplitz matrix whose off-diagonal entries decay smoothly with distance from the diagonal.

This matrix family is convenient for the present experiments for three reasons. First, it is guaranteed to be symmetric positive definite for $0 < \rho < 1$, so both `DSYEV` and `DSYEVD` can be tested on a stable and well-defined class of inputs. Second, the matrix is fully dense, so the experiment exercises the dense reduction, tridiagonal eigensolver, and back-transformation stages that these LAPACK drivers are designed for, without introducing sparsity-specific effects. Third, the single parameter ρ provides a simple way to control how strongly coupled the rows and columns are: choosing $\rho = 0.95$ keeps the matrix far from diagonal, so the eigenvector computation remains nontrivial while the family itself stays easy to describe and reproduce.

Matrix sizes tested:

$$n \in \{512, 1024, 2048, 4096\}.$$

Appendix B gives the detailed matrix specification, illustrative small example, and the residual tests used to validate numerical correctness.

4.2 Overall Performance Benchmark

This section presents the overall performance comparison between `DSYEV` and `DSYEVD` using the optimized OpenBLAS SVE build. This benchmark section therefore measures the SVE-enabled OpenBLAS configuration, not the 128-bit OpenBLAS SIMD baseline used later in Section 4.4. We first report the single-thread baseline and then a small thread-scalability check over 1, 2, and 4 threads. We also report the phase-level runtime breakdown defined in Section 4.1: tridiagonal reduction, tridiagonal eigensolution, and eigenvector back-transformation. The later profiling section uses heavier instrumentation, so small differences in absolute times are expected.

4.2.1 Single-Thread Performance

Table 4.2: Single-thread absolute runtime breakdown for `DSYEV` and `DSYEVD` (JOBZ='V', OpenBLAS SVE, median over repeated runs).

n	Routine	Total (s)	Tridiag. (DSYTRD) (s)	Tri-eig (DSTEQR/DSTEDC) (s)	Back-trans. (DORGTR, DORMTR) (s)
512	<code>DSYEV</code>	0.126	0.013	0.103	0.009
	<code>DSYEVD</code>	0.036	0.013	0.010	0.013
1024	<code>DSYEV</code>	0.932	0.095	0.772	0.064
	<code>DSYEVD</code>	0.239	0.095	0.050	0.094
2048	<code>DSYEV</code>	7.743	0.803	6.439	0.497
	<code>DSYEVD</code>	1.859	0.807	0.294	0.754
4096	<code>DSYEV</code>	66.255	6.574	55.793	3.853
	<code>DSYEVD</code>	14.482	6.616	1.935	5.914

Analysis. Across all sizes, `DSYEVD` is consistently faster than `DSYEV`. The main difference comes from the tridiagonal eigensolver stage. At $n = 4096$, `DSTEQR` accounts

for about 84.2% of `DSYEV` runtime, whereas `DSTEDC` contributes only about 13.4% to `DSYEVD`. By contrast, the dense tridiagonalization stage is nearly the same for the two drivers because both call `DSYTRD`. For the full eigenpair case on this platform, `DSYEVD` is already the faster dense driver once $n \geq 1024$, and the gap widens with problem size. A necessary qualification, developed further in Section 4.6, is that `DSYEVD` attains this speed advantage at the cost of substantially higher workspace demand than `DSYEV`.

4.2.2 Thread Scalability

Table 4.3: Absolute runtime with 1, 2, and 4 threads (OpenBLAS SVE, `JOBZ='V'`, median over repeated runs).

n	Routine	1 Thread [s]	2 Threads [s]	4 Threads [s]
512	<code>DSYEV</code>	0.126	0.118	0.115
	<code>DSYEVD</code>	0.036	0.024	0.017
1024	<code>DSYEV</code>	0.932	0.862	0.830
	<code>DSYEVD</code>	0.239	0.145	0.092
2048	<code>DSYEV</code>	7.743	7.218	6.892
	<code>DSYEVD</code>	1.859	1.106	0.657
4096	<code>DSYEV</code>	66.255	61.823	59.221
	<code>DSYEVD</code>	14.482	8.308	4.773

Table 4.4: Thread speedup and parallel efficiency at $n = 4096$ (OpenBLAS SVE, `JOBZ='V'`). Efficiency is speedup divided by thread count.

Routine	2 Threads		4 Threads	
	Speedup	Efficiency	Speedup	Efficiency
<code>DSYEV</code>	1.07	0.54	1.12	0.28
<code>DSYEVD</code>	1.74	0.87	3.03	0.76

Interpretation. For `DSYEV`, multithread scaling is very limited: even at $n = 4096$, the speedup is only $1.07\times$ from 1 to 2 threads and $1.12\times$ from 1 to 4 threads. In contrast, `DSYEVD` reaches $1.74\times$ and $3.03\times$ over the same thread counts, so its runtime advantage becomes larger on multicore systems. These thread results are secondary confirmation of the single-thread ordering, but they reinforce the same structural point: `DSYEVD` exposes substantially more useful parallel work than `DSYEV`.

4.3 Impact of the Netlib-to-OpenBLAS Implementation Change

This section asks a narrower question than the end-to-end benchmark. The comparison between `DSYEV` and `DSYEVD` is kept fixed, and the software stack is changed from the Netlib reference implementation (Netlib LAPACK + reference BLAS [2, 12, 13]) to the optimized OpenBLAS SVE implementation [14]. Since both the LAPACK and BLAS layers change, this is not a BLAS-only test; the point is to see which stages benefit once the same eigensolver path is moved onto a much stronger implementation stack.

4.3.1 Stage-by-Stage Comparison

Table 4.5 presents the speedup achieved by the OpenBLAS SVE implementation over the Netlib implementation for each computational stage. Speedup is computed as the ratio of Netlib median time to OpenBLAS-SVE median time.

Table 4.5: Speedup of the OpenBLAS SVE implementation over the Netlib implementation by computational stage (single-threaded, speedup = Netlib time / OpenBLAS SVE time).

n	DSYTRD (reduction)	DSTEQR (Tri-eig)	DSTEDC (Tri-eig)	DORGTR (back-trans)	DORMTR (back-trans)
512	2.93	1.00	2.60	5.89	6.37
1024	2.94	1.00	3.48	6.83	7.02
2048	2.76	1.00	4.21	7.21	7.39
4096	2.77	1.00	5.08	7.67	7.77

Interpretation. The stage split in Table 4.5 is very uneven. DSTEQR stays at essentially $\approx 1.0\times$ across all tested sizes, while DSTEDC gains 2.6 to $5.1\times$, DSYTRD gains about 2.8 to $2.9\times$, and the back-transformation stages gain roughly 5.9 to $7.8\times$. In other words, the implementation upgrade helps most where the work is already blocked and BLAS-friendly, and it does almost nothing for the rotation-dominated DSTEQR path. This is why the move from Netlib to OpenBLAS strengthens DSYEVD much more than DSYEV. The next section keeps the OpenBLAS stack fixed and asks the narrower follow-up: how much of the remaining difference can be credited to wider vectors alone?

4.4 Impact of SVE Vectorization

This section keeps the OpenBLAS software stack fixed and changes only the vector-capable kernel target, from a 128-bit OpenBLAS SIMD baseline [14] built for generic ARMV8 to an OpenBLAS build with 256-bit SVE support. The aim is not to revisit the algorithm comparison, but to measure the extra gain available once the blocked OpenBLAS implementation is already in place. In principle, doubling the vector width can double the throughput of a well-vectorized floating-point kernel. In practice, the realized gain depends on how much of each stage is spent in width-sensitive kernels and on whether memory movement or dependencies limit the stage before vector width does.

4.4.1 Stage-by-Stage SIMD Speedup

Table 4.6 presents the speedup achieved by the OpenBLAS SVE build over the OpenBLAS SIMD baseline for each computational stage. Speedup is computed from repeated-run medians under the same single-threaded wrapper configuration.

Table 4.6: Speedup of the OpenBLAS SVE build over the 128-bit OpenBLAS SIMD baseline by stage (single-threaded, speedup = SIMD time / SVE time).

n	DSYTRD (syev)	DSTEQR (Tri-eig)	DSYTRD (syevd)	DSTEDC (Tri-eig)
512	1.59	1.01	1.59	1.07
1024	1.66	1.00	1.71	1.12
2048	1.29	1.00	1.49	1.16
4096	1.43	1.00	1.57	1.18

Interpretation. Table 4.6 shows that wider vectors help, but only within a limited range. DSYTRD gains about 1.3 to 1.7 $\times$, DSTEDC gains only 1.07 to 1.18 $\times$, and DSTEQR stays at $\approx 1.0\times$. The contrast with the previous section is the important point: once the optimized OpenBLAS implementation is already in place, moving from 128-bit SIMD to 256-bit SVE adds a modest extra benefit, but it does not change the qualitative ordering of the stages.

The explanation is consistent with the structure of the routines. Wider vectors help the blocked kernels inside DSYTRD and, to a lesser extent, the DGEMM-dominated path inside DSTEDC. They do not materially help DSTEQR, because that routine is dominated by fine-grained updates with limited SIMD leverage. The role of SVE in this chapter is therefore secondary: it improves the throughput of the existing blocked path, but the larger performance shift has already been created earlier by the algorithmic and implementation changes that made that blocked path available.

4.5 Detailed Performance Profiling

This section provides detailed performance profiling using instrumentation and profiling tools to understand where time is spent within each computational stage. The analysis clarifies the structural basis on which DSTEDC outperforms DSTEQR.

Unless otherwise noted, the percentages in this section come from dedicated profiling runs with finer-grained wrappers, call tracing, and associated logging overhead. They should therefore be read as profile-specific snapshots: small differences relative to the benchmark tables in Section 4.2 are expected and do not change the qualitative conclusions.

Table 4.7: Compact summary of the dominant profiled call paths at $n = 4096$.

Driver path	Stage	Dominant internal path	Share of end-to-end runtime
DSYEV	DSTEQR	DLASR	76.5%
DSYEV	DORGTR	DLARFB $\rightarrow$ DGEMM	6.0%
DSYEV	DSTEDC	DLAEDO $\rightarrow$ DGEMM	10.9%
DSYEV	DORMTR	DORMQL $\rightarrow$ DLARFB $\rightarrow$ DGEMM	38.3%

The detailed inclusive call trees for these four stages are moved to Appendix B.3; the main text keeps only the quantitative points needed for the performance argument.

4.5.1 Analysis of *DSTEQR*

Consistent with the benchmark trend in Section 4.2, *DSYEV* shows that the tridiagonal eigensolve *DSTEQR* dominates the runtime. In the profiling run highlighted here, at $n = 4096$, *DSTEQR* accounts for 82.7% of the total time, followed by the explicit back-transformation *DORGTR* (6.36%) and the tridiagonal reduction *DSYTRD* (10.9%).

Building on that pipeline view, we now instrument the major stages (*JOBZ*='V', dense SPD KMS input [16]) that differentiate *DSYEV* from *DSYEVD*. Top-level timings are reported exclusively per stage, while the breakdowns adopt hierarchical (inclusive) views to reveal dominant call paths.

Focused call flow of DSTEQR. Percentages reported on nodes are shares of the *end-to-end DSYEV* time; the subtree is inclusive.

Key quantitative points. Within *DSTEQR*, *DLASR* alone accounts for 55.6440 s, that is, 92.5% of *DSTEQR* time and 76.5% of end-to-end *DSYEV* time. By contrast, *DLARTG* contributes less than 1% of stage time despite being called more than 12.7 million times, and the remaining named helpers are negligible.

Focused call flow of DORGTR. Percentages are shares of the *DSYEV* end-to-end runtime; the view remains inclusive.

Key quantitative points. The same concentration appears in *DORGTR*. The *DLARFB*→*DGEMM* path accounts for almost the whole stage, with *DGEMM* alone contributing 4.3667 s, or 94.4% of *DORGTR* and 6.00% of end-to-end *DSYEV*. This confirms that the back-transformation path is already efficiently blocked and is not the main leverage point for improving *DSYEV*.

Summary. The profiling evidence isolates the bottleneck in the *DSYEV* path: *DSTEQR* is overwhelmingly dominated by *DLASR*, whereas *DORGTR* is already dispatched through efficient blocked kernels (*DLARFB* to *DGEMM*) and contributes only a small share of the end-to-end runtime. Improving the back-transformation alone therefore cannot materially change the overall behavior of *DSYEV*.

4.5.2 Analysis of *DSTEDC*

Consistent with the benchmark trend in Section 4.2, *DSYEVD* shows a different stage-level distribution compared to *DSYEV*. In the profiling run highlighted here, at $n = 4096$, the tridiagonal reduction *DSYTRD* dominates (45.9%), followed by the back-transformation *DORMTR* (40.4%) and the tridiagonal eigen-solve *DSTEDC* (13.5%). This contrasts sharply with *DSYEV*, where *DSTEQR* accounts for 82.7% of the runtime.

Building on that pipeline view, we now instrument the major stages (*JOBZ*='V', dense SPD KMS input [16]) to reveal their internal structure. Top-level timings are reported exclusively per stage, while the breakdowns of *DSTEDC* and *DORMTR* adopt hierarchical (inclusive) views to highlight dominant call paths and sources of computational intensity.

Focused call flow of DSTEDC. Percentages on nodes are shares of *end-to-end DSYEVD* time; the *DSTEDC* subtree is inclusive by design.

A potentially confusing point is that the *DSTEDC* tree does not show *DSTEQR* explicitly even though LAPACK still uses it at the leaves of the divide-and-conquer recursion. In *DSYEVD*, these calls solve only small tridiagonal subproblems inside *DLAEDO*, so their total cost is minor and they are absorbed into *others* < 1%. This

is fundamentally different from DSYEV, where DSTEQR('V') acts on the full tridiagonal problem and repeatedly updates a dense eigenvector matrix. In DSYEVD, the expensive work is instead shifted to the merge path (DLAED1/DLAED3) and later to DORMTR, which explains why the DSTEDC stage is visible but comparatively small.

Key quantitative points. Within DSTEDC, DLAED0 accounts for 98.4% of stage time, and the merge chain DLAED1/DLAED3 accounts for almost all of that on an inclusive basis. Most importantly, DGEMM alone contributes 1.7606 s, which is 80.6% of DSTEDC time and 10.9% of end-to-end DSYEVD time. By contrast, the remaining named routines inside DSTEDC are individually small, with the next largest term, DLAED4, contributing only 9.4% of stage time.

Focused call flow of DORMTR. We similarly expose the back-transform path; percentages are shares of end-to-end DSYEVD, inclusive within this subtree.

Key quantitative points. The same concentration appears in DORMTR. The DORMQL→DLARFB→DGEMM path is effectively the whole stage, and DGEMM alone contributes 6.1812 s, which is 94.7% of DORMTR time and 38.3% of end-to-end DSYEVD time. The next largest named component, DTRMM, contributes only 2.2% of DORMTR time.

Summary. The profiling evidence for DSYEVD is structurally different from that of DSYEV. Inside DSTEDC, most of the heavy work is funneled through the merge chain and ultimately through DGEMM, while the DORMTR back-transformation is likewise dominated by DLARFB→DGEMM. The remaining non-DGEMM components are individually small, so the main runtime leverage in the DSYEVD path is concentrated in blocked kernels rather than in fine-grained scalar updates. Mechanistically, this happens because DSTEDC merges subproblems by applying eigenvector updates in block form, thereby exposing large DGEMM-dominated work. The detailed trees are collected in Appendix B.3; the main conclusion for the present chapter is simply that the fast DSYEVD path is DGEMM-dominated, whereas the slow DSYEV path is dominated by fine-grained rotation work.

4.5.3 Hardware-Efficiency Evidence

To complement the call-path profiling in Sections 4.5.1 and 4.5.2, we collected direct hardware-counter and memory measurements under the same experimental configuration used in the earlier sections. For small sizes we repeated runs multiple times and report the average per run; for large sizes a single run was used to keep the measurement cost manageable. Since the runtime comparison between DSYEV and DSYEVD has already been established in Section 4.2, the emphasis here is not on repeating the timing comparison, but on addressing a narrower follow-up to Section 4.5.2. Section 4.5.2 identified the fast path as DGEMM-dominated blocked work; the present section asks whether that path also reduces the total hardware cost. The evidence below therefore focuses on cumulative work and memory cost rather than on attributing performance to any one micro-mechanism inside DGEMM.

Table 4.8 isolates the end-to-end memory tradeoff. The faster DSYEVD path consistently requires more workspace, and the ratio approaches 2× for large matrices. This is fully consistent with the design of divide-and-conquer: it accelerates the eigensolve at the cost of substantially larger temporary storage.

Table 4.9 reports the most informative counter-level evidence for the isolated tridiagonal eigensolver stage, DSTEQR executes far more instructions than DSTEDC:

Table 4.8: End-to-end memory footprint of DSYEV and DSYEVD. Peak RSS is reported in MiB.

n	DSYEV RSS [MiB]	DSYEVD RSS [MiB]	Ratio
512	6.6	11.1	$1.69\times$
1024	19.0	35.9	$1.89\times$
2048	67.6	134.1	$1.98\times$
4096	260.6	521.6	$2.00\times$

Table 4.9: Compact hardware evidence for the isolated tridiagonal eigensolver stage. Instruction counts and backend-memory stall counts are reported per run.

n	DSTEQR instr. [$\times 10^9$]	DSTEDC instr. [$\times 10^9$]	Instr. ratio	DSTEQR stalls [$\times 10^9$]	DSTEDC stalls [$\times 10^9$]	Stall ratio
2048	73.52	3.12	$23.60\times$	0.717	0.118	$6.07\times$
4096	602.34	40.28	$14.95\times$	6.043	2.163	$2.79\times$

about $23.6\times$ more at $n = 2048$, $15.0\times$ more at $n = 4096$, respectively. It also accumulates substantially more backend-memory stalls in absolute terms, by factors of $6.1\times$ and $2.8\times$, respectively. These are cumulative costs over the full stage and therefore match the qualitative picture from Sections 4.5.1 and 4.5.2: the implicit QR route spends much more total effort in fine-grained updates, whereas the divide-and-conquer route concentrates more work into fewer, denser kernels. This is the main evidential role of Section 4.5.3: not to establish the internal causes of DGEMM performance in isolation, but to show that the DGEMM-dominated DSTEDC path indeed carries a much smaller total hardware cost than the DSTEQR path.

Normalized counters such as IPC or refill-per-instruction are therefore not the right headline metrics here. DSTEDC can exhibit substantial cache traffic because DGEMM is a high-throughput streaming kernel, yet still finish far earlier because the total amount of work is so much smaller. The stronger conclusion is therefore based on cumulative instruction count and cumulative backend-memory stall cost, not on any single normalized counter in isolation.

For completeness, Tables 4.10 and 4.11 report two normalized deep-counter views of the same isolated tri-eig stage.

Table 4.10: Deep-counter view of the isolated tridiagonal eigensolver: load/store instruction mix. Shares are measured relative to total instructions.

n	DSTEQR load share	DSTEDC load share	DSTEQR store share	DSTEDC store share
1024	0.273	0.274	0.180	0.053
2048	0.273	0.295	0.179	0.037
4096	0.274	0.306	0.175	0.033

These deeper counters refine the interpretation but do not overturn it. Relative to DSTEQR, DSTEDC shows a slightly higher load share and a much lower store share

Table 4.11: Deep-counter view of the isolated tridiagonal eigensolver: L1 load-miss rate and backend-memory stalls per instruction.

n	DSTEQR	DSTEDC	DSTEQR	DSTEDC
	L1 miss rate	L1 miss rate	stall / inst	stall / inst
1024	0.0044	0.0338	0.0069	0.0266
2048	0.0047	0.0783	0.0083	0.0378
4096	0.0188	0.1230	0.0100	0.0578

in Table 4.10, which is consistent with a denser, more streaming-oriented kernel structure. At the same time, Table 4.11 shows that DSTEDC has the higher L1 miss rate and the higher stall-per-instruction value on this platform. This is not a contradiction: a DGEMM-dominated kernel can stream data aggressively and still finish much sooner because the total instruction count and total stall count are far smaller. The cumulative counters in Table 4.9 therefore remain the primary evidence, while the deeper tables mainly support the interpretation of the operation mix.

Overall, the hardware-counter evidence supports the earlier structural interpretation. The main advantage of DSTEDC is that the tri-eig stage does much less total work and organizes that work in a more hardware-friendly form. As a result, DSYEVD is faster than DSYEV because DSTEDC executes far fewer instructions, accumulates less total memory-stall cost, and follows the same blocked-DGEMM-dominated structure exposed by the call-path profiles in Section 4.5.2. The price is a significantly larger workspace, visible in the roughly $2\times$ increase in peak memory usage for DSYEVD in Table 4.8. Read together, Sections 4.5.2 and 4.5.3 therefore establish a clean two-step explanation: first identify that the fast DSYEVD route is DGEMM-dominated blocked work, then show that this route also incurs far less total hardware work than the rotation-dominated DSYEV path.

4.6 Final Synthesis and Implications

This section returns to the main question of the chapter: why is DSYEVD faster than DSYEV in the full-eigenpair setting on the tested platform? [The earlier sections point to one answer with several layers of evidence.](#) The shared reduction stage does not create the difference. The runtime gap appears after reduction, when DSYEV sends the tridiagonal problem to DSTEQR and DSYEVD sends it to DSTEDC.

The stage timings make this visible directly. In the same OpenBLAS SVE backend, the tridiagonal eigensolver stage is $10.38\times$ to $28.88\times$ faster for DSTEDC than for DSTEQR. The $n = 4096$ case makes the attribution especially clear: end-to-end time falls from 66.255 s for DSYEV to 14.482 s for DSYEVD, while the shared DSYTRD stage stays almost unchanged at 6.574 s and 6.616 s. The observed speed gap is therefore created almost entirely in the tridiagonal eigenvector path.

Profiling and counter evidence then explain why that stage behaves so differently. In the DSYEV path, DSTEQR is dominated by DLASR and long chains of fine-grained rotation updates. In the DSYEVD path, the expensive work is reorganized into the DLAED* merge chain and then into blocked kernels dominated by DGEMM; at $n = 4096$, DGEMM contributes 10.9% of end-to-end time inside DSTEDC and 38.3% inside DORMTR.

This is also why the implementation and SIMD comparisons line up the way they do. The move from Netlib to OpenBLAS gives a large gain to the blocked stages, whereas DSTEQR stays near $1.0\times$. The later move from 128-bit SIMD to 256-bit SVE adds only a smaller extra benefit, with DSTEDC gaining 1.07 to $1.18\times$. The order of importance is therefore clear: algorithmic restructuring first, implementation quality second, vector width third.

That conclusion does not make DSYEVD unconditionally better. The same experiments show a clear memory cost, and that cost grows with problem size. The divide-and-conquer path buys runtime by using substantially more workspace.

Table 4.12: Summary comparison between DSYEV and DSYEVD for the full eigenpair case, with representative $n = 4096$ runtime and memory values.

Aspect	DSYEV	DSYEVD
Tri-eig routine	DSTEQR (implicit QR iteration)	DSTEDC (divide-and-conquer)
Main work pattern	Rotation-heavy updates that are hard to accelerate	Blocked merge/update path dominated by DGEMM
Total runtime at $n = 4096$	66.255 s	14.482 s
Peak memory at $n = 4096$	260.6 MiB	521.6 MiB
Primary advantage	Lower workspace demand	Much faster full eigenpair computation
Primary limitation	Tri-eig stage dominates the end-to-end cost	Roughly $2\times$ higher peak memory
More appropriate when	Memory pressure matters more than runtime	Runtime matters and workspace is available

Runtime values come from Section 4.2. Peak memory comes from Section 4.5.3.

Table 4.12 summarizes the main tradeoff of this chapter in compact form. DSYEV and DSYEVD mainly differ in the tridiagonal eigensolver they use, and that design choice propagates to the hot path, total runtime, and memory demand. At $n = 4096$, DSYEVD is $4.57\times$ faster overall, while its peak RSS is approximately $2\times$ higher. The detailed instruction and stall-count evidence remains in Section 4.5.3; the table here is limited to the quantities most directly relevant to the comparison between the two drivers.

Within the KMS/OpenBLAS/ARM setting of this chapter, the practical reading is straightforward. For full eigenpairs, DSYEVD is the better choice when runtime matters more than memory. DSYEV remains relevant when workspace pressure matters more, or when the absolute runtime difference is not decisive for the application. The structural lesson is likely more portable than the exact numerical ratios: the advantage of DSYEVD appears when the tridiagonal work can be reorganized into blocked kernels that the software and hardware stack execute efficiently.

CHAPTER 5

Conclusion and Future Work

5.1 Main Findings

This thesis set out to answer a fairly specific practical question: why do the two LAPACK drivers `DSYEV` and `DSYEVD`, which solve the same dense symmetric eigenproblem, behave so differently once eigenvectors are requested? The comparison in Chapters 2–4 shows that the answer is not hidden in the shared dense reduction stage, but in what happens after that stage. Both drivers call `DSYTRD`, so their tridiagonalization cost is almost the same. The decisive split is that `DSYEV` sends the tridiagonal problem to `DSTEQR`, whereas `DSYEVD` sends it to `DSTEDC`.

In the experiments of Chapter 4, that design choice dominates the runtime comparison. For the full eigenpair case with the OpenBLAS SVE build, `DSYEV` takes 66.255 s at $n = 4096$, while `DSYEVD` takes 14.482 s. The reduction stage is essentially unchanged (6.574 s versus 6.616 s), so almost all of the gap is created in the tridiagonal eigensolver: 55.793 s for `DSTEQR` against 1.935 s for `DSTEDC`. The same pattern is already visible at $n = 1024$ and becomes stronger as n grows. For this thesis, that is the main empirical result: the practical advantage of `DSYEVD` is created by reorganizing the tridiagonal eigenvector computation, not by making the shared parts of the pipeline slightly faster.

The profiling and hardware-counter evidence makes that statement more precise. On the `DSYEV` side, the slow path is dominated by the rotation-heavy `DSTEQR` routine, and within that routine `DLASR` alone accounts for 76.5% of the end-to-end runtime in the $n = 4096$ profile. On the `DSYEVD` side, the heavy work is pushed into blocked kernels: `DGEMM` contributes 10.9% of end-to-end time inside `DSTEDC` and 38.3% inside `DORMTR`. The counter data points in the same direction. At $n = 4096$, the isolated tridiagonal eigensolver stage of `DSTEQR` executes about $15.0\times$ as many instructions as `DSTEDC`, together with about $2.8\times$ as many backend-memory stalls. The faster driver is therefore not simply the one with better BLAS tuning; it is the one whose dominant work has already been reorganized into a form that optimized BLAS can use well.

That said, the thesis does not support a universal ranking of the two drivers. The measured speed advantage of `DSYEVD` comes with a clear workspace cost: peak RSS rises from 260.6 MiB for `DSYEV` to 521.6 MiB for `DSYEVD` at $n = 4096$, and the ratio is already close to $2\times$ by $n = 2048$. In other words, the practical conclusion is not that `DSYEV` has become obsolete. It is that, within the full-eigenpair setting studied here, `DSYEVD` buys runtime by spending more workspace.

5.2 Practical Guidance for Routine Selection

The practical recommendation of this thesis should be read with the scope of Chapter 4 in mind. The detailed experiments are for the full eigenpair case ($\text{JOBZ}='V'$) on dense SPD KMS matrices, using LAPACK backed by the tested OpenBLAS build on a single-node ARM platform. Within that setting, `DSYEVD` is the routine to choose when runtime is the main priority.

The numerical reason is already visible in the stage timings. Once eigenvectors are requested, the two drivers pay almost the same cost for `DSYTRD`, but they do not pay the same cost for the tridiagonal eigensolve. At $n = 1024$, the total time drops from 0.932 s for `DSYEV` to 0.239 s for `DSYEVD`; at $n = 4096$, it drops from 66.255 s to 14.482 s. The thread experiment points the same way: at $n = 4096$, `DSYEV` gains only $1.12\times$ from 1 to 4 threads, while `DSYEVD` reaches $3.03\times$. For a workflow that repeatedly computes full eigenpairs of moderately large dense matrices, these are not marginal differences.

`DSYEV` still has a place, but the reason to keep it is now clearer. Its main advantage in this comparison is lower workspace demand, not better runtime. If memory pressure is tight, if the matrices are small enough that the absolute timing difference is unimportant, or if a code path already relies on the lighter-weight QR driver, then `DSYEV` remains a defensible choice. The decision is therefore best viewed as a runtime–workspace tradeoff rather than a winner-takes-all ranking.

The eigenvalues-only case should be interpreted more cautiously. The brief introductory benchmark in Chapter 1 suggests that the timing gap is much smaller when eigenvectors are not formed, which is consistent with the structure of the two drivers. However, the detailed analysis in this thesis was designed around the full eigenpair path, because that is where the structural difference between `DSTEQR` and `DSTEDC` matters most.

For any future comparison, the experimental controls used here are worth keeping: fix the matrix family, thread count, BLAS/LAPACK backend, and problem sizes before comparing routines. Otherwise, it becomes difficult to tell whether an observed speedup is caused by the algorithmic path or by changes in the software and hardware environment.

5.3 Limitations of This Study

The conclusions of this thesis are strongest inside the experimental frame that was chosen to make the comparison interpretable. They should not be read as a claim about every dense symmetric eigenproblem or every LAPACK deployment.

The first boundary is the software-hardware stack. The measurements were taken for LAPACK drivers backed mainly by OpenBLAS on the tested ARM CPU platform. That stack is appropriate for the implementation questions raised in this thesis, but the exact timing ratios can change under MKL, BLIS, vendor-tuned libraries, or different processors. The structural argument should carry over more widely than the raw numbers, yet the raw numbers themselves belong to this particular environment.

The second boundary is the matrix model. Chapter 4 deliberately uses dense SPD KMS matrices with a fixed correlation parameter. The tested sizes are 512, 1024, 2048, and 4096. This keeps the phase-level comparisons clean, but it does not

represent the full range of spectra and matrix families seen in applications. Matrices with different clustering patterns, conditioning, or application-specific structure may shift the detailed balance between the stages, even if the overall algorithmic picture remains similar.

The third boundary is that the thesis concentrates on single-node CPU execution. The thread study checks 1, 2, and 4 threads on one machine, but it does not extend to GPUs, heterogeneous CPU–GPU workflows, or distributed memory. Those settings matter in modern scientific computing, especially once matrix sizes exceed the comfortable range of a single shared-memory node.

The fourth boundary is the choice of performance criteria. The analysis is deliberately runtime-centered, with supporting discussion of workspace and hardware counters. Accuracy checks were used to validate the experiments, but the thesis does not build a full comparative study of numerical robustness, energy use, or joint runtime–memory efficiency across many platforms. For that reason, the routine-selection advice in this chapter should be read mainly as runtime guidance under the tested conditions.

5.4 Future Work

The most immediate extension is to keep the same measurement framework and move it across libraries and machines. Repeating the Chapter 4 pipeline analysis for OpenBLAS, MKL, BLIS, and vendor-tuned ARM libraries would show more clearly which parts of the observed speedup are robust consequences of the DSTEQR versus DSTEDC choice and which parts depend on a particular backend.

A second extension is to stress the routines with a wider set of inputs. The current KMS benchmark was useful because it gave a controlled comparison, but a broader study should add matrix families with different spectral clustering, different conditioning, and perhaps non-Toeplitz structure. That would test how stable the present runtime ordering remains once the tridiagonal problems become less uniform.

A third extension is to make the tradeoff analysis more complete. The present thesis already shows that DSYEVD can save a large amount of runtime while using about twice the memory for the tested large cases. A next step would be to measure runtime, workspace, numerical residuals, and energy use together, so that routine selection is informed by more than runtime alone.

Beyond that, the same questions can be asked in settings that this thesis left aside, including GPUs, heterogeneous nodes, and distributed-memory implementations. Those environments may change the practical balance again, especially because the blocked work that benefits DSYEVD on a multicore CPU may map even more naturally to accelerator-oriented architectures.

APPENDIX A

Summary of Main LAPACK Routines

This appendix collects the small set of LAPACK routines that appear repeatedly in the implementation discussion of Chapters 2 and 3. The purpose is not to replicate LAPACK documentation, but to record the role each routine plays in the present thesis.

Table A.1: Main LAPACK routines discussed in the thesis.

Routine	Role in this thesis	Used in
DSYTRD	Reduces a dense symmetric matrix to tridiagonal form and stores the Householder reflectors needed for later eigenvector reconstruction.	DSYEV, DSYEVD
DSTEQR	Solves the symmetric tridiagonal eigenproblem by implicit QR/QL iteration; in the DSYEV path it also updates eigenvectors through many sequential rotations.	DSYEV
DSTEDC	Solves the symmetric tridiagonal eigenproblem by divide-and-conquer, organizing the dominant work into recursive subproblems and blocked merge operations.	DSYEVD
DORGTR	Forms the orthogonal matrix Q explicitly from the reflectors stored during DSYTRD.	DSYEV
DORMTR	Applies the stored orthogonal transformations to map tridiagonal eigenvectors back to the eigenvectors of the original dense matrix.	DSYEVD

APPENDIX B

Detailed Experimental Setup

This appendix records the experimental details that support the main results in Chapter 4 but are not essential to keep in the chapter’s main flow.

B.1 Hardware and Software Configuration

All experiments were run on an AWS EC2 `c7g.xlarge` instance equipped with an AWS Graviton3 processor (ARMv8-A with SVE support). Linux reports a default SVE vector length of 32 bytes, that is, 256 bits. The machine characteristics are:

- **CPU:** 4 vCPUs, single hardware thread per core
- **Memory:** 8 GiB RAM
- **Cache hierarchy:** 64 KiB L1 data cache and 64 KiB L1 instruction cache per core, 1 MiB L2 cache per core, and a shared 32 MiB L3 cache

All code was compiled using GCC with the following optimization flags:

```
-O3 -std=c11 -mcpu=native -mtune=native
-fno-math-errno -fno-trapping-math -ffp-contract=fast
```

Chapter 4 compares three library configurations: Reference LAPACK with Reference BLAS, Reference LAPACK with OpenBLAS SIMD, and Reference LAPACK with OpenBLAS SVE; see Table 4.1. The `OpenBLAS SIMD` build targets generic ARMV8 kernels without SVE, whereas the `OpenBLAS SVE` build targets the Graviton3-capable `NEOVERSEV1` backend. For the benchmark runs in Section 4.2, the active OpenBLAS SVE library is a pthread build with `MAX_THREADS=8`. Since the machine has 4 vCPUs, the explicit thread-scaling experiment is limited to 1, 2, and 4 threads.

The controlled comparisons in Sections 4.3 and 4.4 were run in single-threaded mode by setting

- `OMP_NUM_THREADS=1`
- `OPENBLAS_NUM_THREADS=1`
- `ARMPL_NUM_THREADS=1`

This keeps algorithmic and library-level effects separate from thread scheduling.

B.2 Measurement Protocol

The chapter uses three related measurement modes:

- the benchmark tables in Section 4.2, which report end-to-end wall-clock time together with lightweight stage timers;
- the controlled library-stack and SIMD comparisons in Sections 4.3 and 4.4, which keep the matrix family and threading configuration fixed;

- the deeper profiling and hardware-counter experiments, which use dedicated binaries with heavier instrumentation and are interpreted as explanatory evidence rather than exact timing replicas.

For the benchmark and controlled-comparison tables in Sections 4.2–4.4, each configuration first executes one warm-up run that is discarded, then repeated measured runs with the same wrapper configuration. We use five measured runs for $n \leq 2048$ and three measured runs for $n = 4096$. The tables in the thesis report the median runtime of those measured runs, while the corresponding CSV outputs in the accompanying repository [15] additionally store the mean, sample standard deviation, minimum, maximum, and repeat count. This choice reduces sensitivity to occasional cloud-runtime jitter while preserving the raw variability information for reproducibility.

Timing measurements were obtained using wrapper functions that instrument the main LAPACK stages at the subroutine level. This provides separate timings for DSYTRD, for the tridiagonal eigensolver (DSTEQR or DSTEDC), and for the final eigenvector mapping (DORGTR or DORMTR).

B.3 Additional Profiling Trees

The main text in Section 4.5 reports only the quantitative conclusions from the profiling runs. The detailed inclusive call-tree views are collected here for reference.

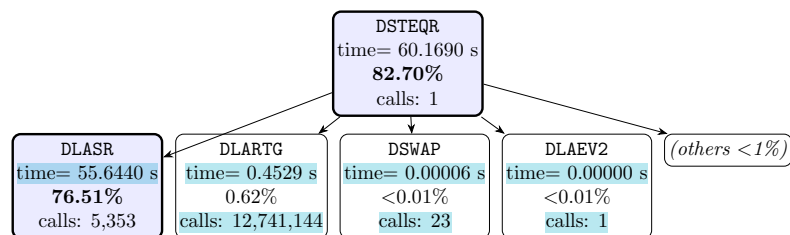


Figure B.1: Hot-path hierarchy inside DSTEQR (inclusive).

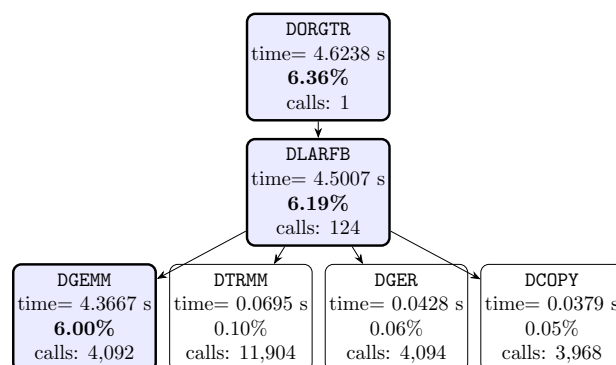


Figure B.2: Hot-path hierarchy inside DORGTR (inclusive).

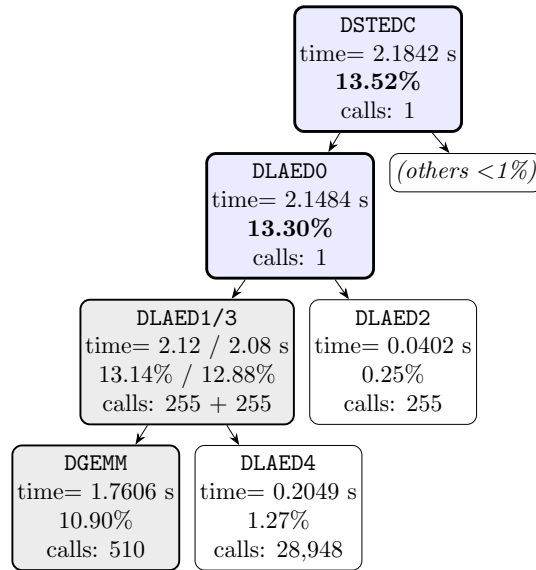


Figure B.3: Hot-path hierarchy inside DSTEDC (inclusive). Node labels show time and relative share; leaf-level DSTEDC calls are grouped into *others* < 1%.

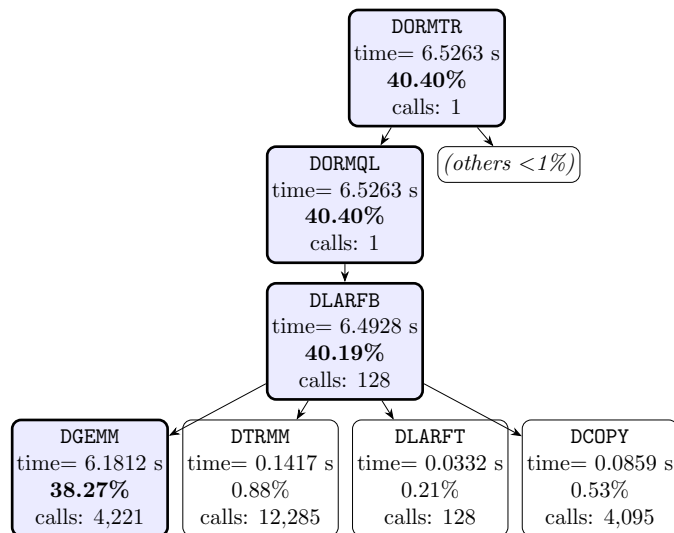


Figure B.4: Hot-path hierarchy inside DORMTR (inclusive). Time and relative share are shown per node.

B.4 Test Matrices and Correctness Checks

All experiments use Symmetric Positive Definite (SPD) Kac–Murdock–Szegő (KMS) matrices with correlation parameter $\rho = 0.95$, defined by

$$A_{ij} = \rho^{|i-j|}, \quad 0 < \rho < 1.$$

This gives a dense symmetric Toeplitz matrix whose off-diagonal entries decay smoothly with distance from the diagonal. The tested matrix sizes are

$$n \in \{512, 1024, 2048, 4096\}.$$

For illustration, when $n = 5$ and $\rho = 0.95$,

$$\begin{bmatrix} 1 & 0.95 & 0.9025 & 0.8574 & 0.8145 \\ 0.95 & 1 & 0.95 & 0.9025 & 0.8574 \\ 0.9025 & 0.95 & 1 & 0.95 & 0.9025 \\ 0.8574 & 0.9025 & 0.95 & 1 & 0.95 \\ 0.8145 & 0.8574 & 0.9025 & 0.95 & 1 \end{bmatrix}.$$

Numerical correctness was verified by computing the standard LAPACK residuals

$$\frac{\|AV - V\Lambda\|_F}{n\|A\|_F}, \quad \|V^\top V - I\|_F,$$

which remained within recommended thresholds for all reported experiments.

References

- [1] R.S. Martin, C. Reinsch, and J.H. Wilkinson. Householder's tridiagonalization of a symmetric matrix. *Numerische Mathematik*, 1968.
- [2] E. Anderson, Z. Bai, C. Bischof, L. S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen. *LAPACK Users' Guide*. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, 3rd edition, 1999.
- [3] J. J. M. Cuppen. A divide and conquer method for the symmetric tridiagonal eigenproblem. *Numerische Mathematik*, 36(2):177–195, 1981.
- [4] Ming Gu and Stanley C. Eisenstat. A divide-and-conquer algorithm for the symmetric tridiagonal eigenproblem. *SIAM Journal on Matrix Analysis and Applications*, 16(1):172–191, 1995.
- [5] Jack J. Dongarra and Danny C. Sorensen. A fully parallel algorithm for the symmetric eigenvalue problem. *SIAM Journal on Scientific and Statistical Computing*, 8(2):s139–s154, 1987.
- [6] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.
- [7] Beresford N. Parlett. *The Symmetric Eigenvalue Problem*. SIAM, classics in applied mathematics edition, 1998.
- [8] David S. Watkins. Understanding the qr algorithm. *SIAM Review*, 44(1):84–101, 2002.
- [9] J. G. F. Francis. The qr transformation: A unitary analogue to the lr transformation. *The Computer Journal*, 4(3):265–271, 1961.
- [10] J. G. F. Francis. The qr transformation, part 2. *The Computer Journal*, 4(4):332–345, 1962.
- [11] J. H. Wilkinson. *The Algebraic Eigenvalue Problem*. Clarendon Press, Oxford, 1965.
- [12] Netlib. Lapack – linear algebra package. <https://www.netlib.org/lapack/>, 2026. Reference LAPACK distribution, accessed March 13, 2026.
- [13] Netlib. Blas (basic linear algebra subprograms). <https://www.netlib.org/blas/>, 2026. Reference BLAS distribution, accessed March 13, 2026.
- [14] OpenBLAS Developers. Openblas. <https://github.com/OpenMathLib/OpenBLAS>, 2026. Optimized BLAS library based on GotoBLAS2, accessed March 13, 2026.
- [15] Yin, Ming. Math5005-5006_mingyin. https://github.com/yinmingjiushang/MATH5005-5006_mingyin, 2026. Repository containing the Chapter 4 experiment code and result snapshots, accessed March 16, 2026.
- [16] M. Kac, W. L. Murdock, and G. Szegő. On the eigenvalues of certain hermitian forms. *Journal of Rational Mechanics and Analysis*, 2:767–800, 1953.

Acknowledgement of AI Use

I acknowledge the assistance provided by OpenAI Codex for text and code enhancement. I confirm that once text or code was modified with AI assistance, I checked it for possible errors, inaccuracies, harmful content, and bias. I made changes where necessary to ensure that the submitted work, including its voice, arguments, and conclusions, is my own within the parameters of permitted AI use.

The use of AI in my work includes:

- paraphrasing and language editing;
- improving the organisation, structure, and coherence of the thesis;
- helping form and refine ideas for presenting mathematical exposition and improving the flow of arguments;
- helping explore implementation details relevant to LAPACK routines;
- providing programming support, including script writing, code refinement, code comments, and preparing clean copies of code and experimental templates;
- suggesting improvements to the design and presentation of experiments, including robustness and stability considerations;
- assisting with LaTeX drafting and typesetting based on author-provided content.

AI-assisted text and code were developed through iterative prompting and revision. This included revising prompts, reviewing generated output, comparing alternatives, and refining the final wording or implementation before inclusion in the thesis or supporting code. Representative prompts included “improve the coherence of this paragraph”, “suggest a clearer structure for this chapter”, “clarify the transition in this mathematical explanation”, “help explore relevant LAPACK implementation details”, “add comments to this script”, “prepare a clean copy of this experimental code template”, “help organise these experimental comparisons”, and “suggest additional robustness or stability checks for this experiment”.

All mathematical arguments, technical verification, implementation choices, experimental work, interpretation of results, and final conclusions were independently reviewed by the author. The author takes full responsibility for the final submitted thesis.